

KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
REPUBLIK INDONESIA, 2024
Informatika untuk SMA/MA Kelas XI
Penulis : Dela Chaerani, dkk.
ISBN : 978-623-388-204-0 (jil.2 PDF)

Bab 4

Pengembangan Kemampuan Berpikir Komputasional dan Implementasi Algoritma



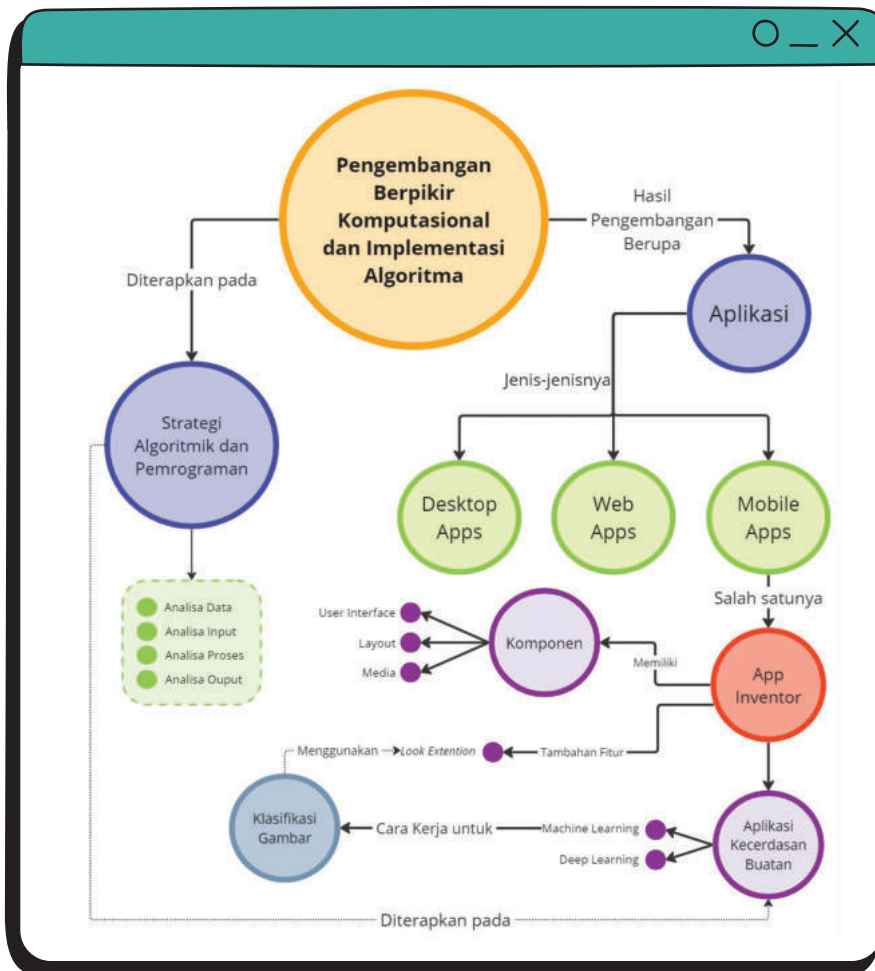


Tujuan Pembelajaran

Setelah mempelajari bab ini kamu akan mampu menerapkan konsep strategi algoritmik, menggunakan dan mengembangkan program komputer terstruktur dalam notasi algoritma dan melakukan perancangan algoritma dan mengimplementasikannya.



Peta Konsep



Gambar 4.1 Peta Konsep Pembelajaran Pengembangan Berpikir Komputasional & Implementasi Algoritma



Kata Kunci

- Aplikasi *Mobile*
- Kecerdasan Buatan
- *Machine Learning*
- Klasifikasi



Siap-Siap Belajar

Hakikatnya kita sebagai manusia memiliki akal untuk berpikir dalam menyelesaikan masalah dan dapat terus meningkatkan kecerdasan kita, lalu bagaimana dengan sebuah mesin buatan manusia? Dapatkah mesin tersebut berpikir dan terus meningkatkan kecerdasan berpikirnya seperti manusia?



Apersepsi

Saat ini mungkin sebagian dari kamu telah terbiasa menggunakan gawai (*mobile phone*) berupa ponsel, tablet, atau yang lain. Mungkin sebagian dari kamu juga telah terbiasa menggunakan aplikasi *mobile*. Aplikasi *mobile* adalah salah satu bentuk artefak komputasional yang bermanfaat bagi kehidupan masyarakat di era digital saat ini. Aplikasi ini sama dengan aplikasi lain yang telah kamu kembangkan pada jenjang sebelumnya namun dirancang untuk dapat berjalan pada ponsel (*mobile phone*). Pengembangan aplikasi *mobile* tidaklah sesulit yang dibayangkan. Pada bab ini kamu akan mempelajari bagaimana cara mengembangkan aplikasi *mobile*, dan dilanjutkan dengan penggunaan *library* atau komponen Kecerdasan Buatan. *Library* adalah modul program dengan fungsi tertentu yang sudah dikemas sehingga siap dipakai tanpa pemrogram pemakainya perlu mengimplementasi kodenya. Memakai *library* ini dapat diibaratkan kamu menggunakan ponsel atau komputer dengan mudah dan nyaman tanpa perlu tahu betapa rumit isi di dalamnya.

A. Menerapkan Strategi Algoritmik dan Pemrograman

Pada bagian ini kamu akan membuat berbagai program berdasarkan permasalahan yang tersedia. Tiap permasalahan memiliki sub permasalahannya tersendiri yang tingkat kesulitannya meningkat. Kamu akan membuat program dimulai dari perancangan, yaitu merancang algoritma untuk menyelesaikan permasalahan tersebut. Selanjutnya, algoritma tersebut kamu terjemahkan ke dalam bahasa pemrograman yang kamu kuasai, misalnya bahasa C, Python. Permasalahan tersebut akan meningkatkan kemampuan programming kamu dengan mempelajari bagian ini dengan menyelesaikan berbagai sub permasalahan yang tersedia.

1. Problem Simulasi Burung

Pada bagian ini, kamu akan membuat program untuk mensimulasikan gerak burung yang diluncurkan dengan menggunakan alat ketapel. Secara prinsip, gerakan burung yang diluncurkan dengan menggunakan ketapel menggunakan prinsip gerak lurus berubah beraturan (GLBB). Terdapat komponen sudut, gravitasi serta kecepatan dan waktu yang menjadi penentu jauhnya burung tersebut dapat meluncur dengan menggunakan ketapel. Konsep ini menggunakan kaidah gerak parabola yang telah kamu pelajari pada mata pelajaran Fisika. *Problem* akan dibagi menjadi beberapa *subproblem* dengan tingkat kesulitan yang meningkat. Ikutilah petunjuk gurumu dalam memilih tingkat kesulitan *subproblem* yang akan kamu kerjakan. Apabila kamu berhasil mengerjakan *subproblem* tersebut, kamu dapat menantang diri kamu untuk mengerjakan *subproblem* yang lebih sulit.

Setelah menentukan tingkat kesulitan *subproblem* yang akan dikerjakan, kamu dapat mulai mengerjakan aktivitas Ayo Merancang Program: Merancang Algoritma Simulasi Burung dan Ayo Buat Program: Membuat Program Simulasi Burung berdasarkan deskripsi permasalahan yang sesuai.

Problem: Program Simulasi Burung

Boro adalah seekor burung yang terjatuh dari sarangnya di sebuah pohon saat sedang tidur. Saat terjatuh, sayap Boro menghantam tanah dan ia sangat merasa kesakitan. Boro harus kembali ke sarangnya. Namun, karena sayapnya terluka, ia tidak dapat terbang sebagaimana mestinya. Tak jauh dari tempat

Boro jatuh, terdapat ketapel raksasa yang biasa digunakan oleh pemilik lahan untuk kegiatan sirkus. Ketapel tersebut biasa digunakan untuk kegiatan menembak dengan menggunakan buah semangka.



Gambar 4.2 Ilustrasi Boro sedang Terjatuh

Kebetulan saat itu, pemilik lahan sedang latihan sirkus menembak semangka dengan menggunakan ketapel. Boro meminta tolong kepada pemilik lahan, apakah ia dapat ikut di atas semangka tersebut agar dapat kembali ke rumahnya di atas pohon. Sang pemilik lahan setuju akan membantu dan menanyakan kepada Boro tentang lokasi pohon tempat tinggalnya.

Subproblem 1: Menghitung Jarak Horizontal Terjauh

Deskripsi:

Pada *subproblem* ini, kamu akan diminta untuk menghitung jarak horisontal terjauh yang dapat ditempuh oleh Boro apabila Boro ikut berdiri di atas semangka yang akan diluncurkan oleh pemilik lahan.

Format Masukan:

Baris pertama adalah sebuah bilangan bulat S yang menggambarkan sudut peluncuran. Nilai S ini bernilai $0-90$. Baris kedua adalah V yang merupakan kecepatan awal Boro saat meluncur dengan menggunakan ketapel. Asumsikan bahwa nilai gravitasi adalah 10 .

Format Keluaran:

Keluaran berupa bilangan yang menunjukkan jarak terjauh Boro mendarat di tanah.

Tabel 4.1 Format Keluaran Program Simulasi Burung *Subproblem 1*

Contoh Masukan	Contoh Keluaran
37 10	9.6

Subproblem 2: Menghitung Waktu

Deskripsi:

Pada *subproblem* ini, kamu akan diminta untuk menghitung waktu yang diperlukan Boro untuk mencapai jarak horizontal terjauh apabila Boro ikut berdiri di atas semangka yang akan diluncurkan oleh pemilik lahan.

Format Masukan:

Baris pertama adalah sebuah bilangan bulat S yang menggambarkan sudut peluncuran. Nilai S ini bernilai $0-90$. Baris kedua adalah V yang merupakan kecepatan awal Boro saat meluncur dengan menggunakan ketapel. Asumsikan bahwa nilai gravitasi adalah 10 .

Format Keluaran:

Keluaran berupa bilangan yang menunjukkan waktu yang ditempuh Boro untuk mencapai jarak terjauh dengan format 3 angka di belakang koma.

Tabel 4.2 Format Keluaran Program Simulasi Burung *Subproblem 2*

Contoh Masukan	Contoh Keluaran
37 100	12.036

Subproblem 3: Prediksi Ketinggian Dicapai Boro

Deskripsi:

Pada *subproblem* ini, kamu akan diminta untuk memberikan prediksi apakah Boro dapat mencapai ketinggian lebih tinggi daripada tinggi pohon tempat ia bersarang ketika Boro ikut berdiri di atas semangka yang akan diluncurkan oleh pemilik lahan.

Format Masukan:

Baris pertama adalah sebuah bilangan bulat S yang menggambarkan sudut peluncuran. Nilai S ini bernilai 0–90. Baris kedua adalah V yang merupakan kecepatan awal Boro saat meluncur dengan menggunakan ketapel. Baris ketiga adalah T yang merupakan tinggi pohon tempat Boro Bersarang. Asumsikan bahwa nilai gravitasi adalah 10.

Format Keluaran:

Keluaran berupa bilangan yang menunjukkan status ketinggian Boro dibandingkan dengan tinggi pohon tempat sarang Boro: 1 apabila Boro dapat mencapai ketinggian sama dengan tinggi pohon tempat sarangnya berada atau lebih; 0 apabila Boro tidak mampu mencapai ketinggian yang sama dengan pohon tersebut. Keluaran juga berupa ketinggian maksimum yang dapat diperoleh oleh Boro saat meluncur dengan menggunakan ketapel.

Tabel 4.3 Format Keluaran Program Simulasi Burung *Subproblem 3*

Contoh Masukan	Contoh Keluaran
37 100 100	Status : 1 Ketinggian : 181.09
37 100 200	Status : 0 Ketinggian : 181.09

Subproblem 4: Prediksi Ketinggian Dicapai Boro dan Teman-temannya

Deskripsi:

Pada *subproblem* ini, kamu akan diminta untuk memberikan prediksi apakah Boro dan teman-temannya dapat mencapai ketinggian lebih tinggi daripada tinggi pohon tempat ia bersarang. Pada *subproblem* ini, burung yang akan menumpang di atas semangka tidak hanya Boro, tetapi terdapat burung lain yang akan menumpang secara bergantian.

Format Masukan:

Baris pertama adalah sebuah bilangan bulat N yang menggambarkan jumlah burung yang akan ikut peluncuran. Baris kedua adalah S yang menggambarkan sudut peluncuran. Nilai S ini bernilai 0-90. Baris ketiga adalah V yang merupakan kecepatan awal burung saat meluncur dengan menggunakan ketapel. Baris keempat adalah T yang merupakan tinggi pohon tempat burung bersarang. Asumsikan bahwa nilai gravitasi adalah 10.

Format Keluaran:

Keluaran berupa urutan burung dan bilangan yang menunjukkan status ketinggian burung dibandingkan dengan tinggi pohon tempat sarang burung tersebut: 1 apabila burung dapat mencapai ketinggian sama dengan tinggi pohon tempat sarangnya berada atau lebih, 0 apabila burung tidak mampu mencapai ketinggian yang sama dengan pohon tersebut. Keluaran juga berupa ketinggian maksimum yang dapat diperoleh oleh burung saat meluncur dengan menggunakan ketapel.

Tabel 4.4 Format Keluaran Program Simulasi Burung *Subproblem 4*

Contoh Masukan	Contoh Keluaran
1 37	Status Burung 1 : 1 Ketinggian: 181.09
100 100	Status Burung 1 : 1 Ketinggian: 181.09
2 37 100 100 37 100 200	Status Burung 2 : 0 Ketinggian: 181.09



Ayo Merancang Program



Merancang Algoritma Simulasi Burung

Jenis Aktivitas: Berpasangan

No Aktivitas: LD-K11-01

Berdasarkan deskripsi permasalahan di atas, secara individu, definisikanlah permasalahan dan rancanglah algoritma solusi dari permasalahan tersebut. Kamu dapat membuka kembali bahan belajar tentang simulasi burung yang menjadi domain permasalahan yang diberikan. Dokumentasikanlah setiap langkah yang kamu kerjakan, termasuk apa yang kamu hasilkan dalam buku kerja kamu.

Setelah kamu selesai merancang algoritma, tukarkan algoritma yang kamu punya kepada pasanganmu.. Setelah itu, telusurilah algoritma temanmu dan cek apakah algoritma tersebut sudah benar atau belum. Apabila belum benar, diskusikanlah apa yang dapat diperbaiki dari rancangan algoritma tersebut bersama-sama. Jangan lupa untuk membandingkan solusi yang telah kamu hasilkan. Apabila solusi kamu berbeda, tetapi sama-sama menghasilkan jawaban yang benar, bandingkanlah kedua solusi tersebut.



Ayo Merancang Program

Membuat Program Simulasi Burung

Jenis Aktivitas: Berpasangan

No Aktivitas: LD-K11-02

Sekarang, secara individu, implementasikanlah algoritma yang telah kamu rancang dalam bentuk program dengan menggunakan bahasa pemrograman yang telah kamu kuasai. Sebelum melakukan kompilasi program, tukarkan kode programmu dengan kode temanmu dan cek apakah kode program tersebut sudah ditulis dengan benar. Setelah itu, lakukan kompilasi kode tersebut menjadi program, lalu ujilah program temanmu dengan kasus uji yang kamu rancang. Apabila program teman kamu belum menghasilkan jawaban yang benar, sampaikanlah kepada

temanmu untuk memperbaiki kode program tersebut hingga menghasilkan jawaban yang benar. Setelah selesai, presentasikanlah hasil kerjamu di depan kelas, mengikuti petunjuk dari guru.

2. Problem Pengelolaan Bank Darah

Pada bagian ini, kamu akan menyelesaikan suatu permasalahan tentang golongan darah. Informasi mengenai golongan darah sangat berguna di bidang kesehatan, bahkan dapat menyelamatkan nyawa seseorang. Dalam *problem* ini, kamu akan membantu sebuah rumah sakit untuk memastikan apakah stok darah yang mereka miliki cukup.

Problem: Pengelolaan Bank Darah

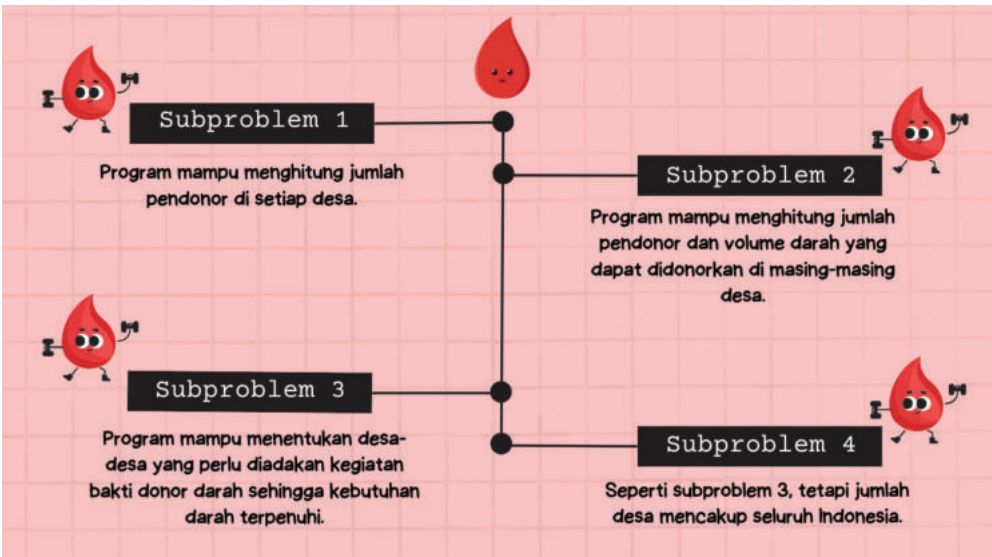
Suatu bank darah di rumah sakit mengelola stok darah yang mereka miliki. Persediaan darah yang mereka miliki dikelompokkan berdasarkan golongan darah standar A, AB, B, dan O seperti yang selama ini kita kenal. Kondisi stok darah yang aman adalah tersedianya cadangan yang cukup untuk setiap golongan darah. Apabila terdapat golongan darah yang stoknya menipis atau habis, pihak Bank Darah akan melakukan kegiatan bakti donor darah di suatu desa. Masalah ini di dunia nyata lebih kompleks karena mempertimbangkan rhesus darah dan faktor lainnya, tetapi kita sederhanakan dengan hanya mempertimbangkan golongan darah pendonor.

Agar kegiatan bakti donor darah dapat berjalan secara efektif dan tepat sasaran, pihak Bank Darah mencatat data para Donor Darah Sukarela (DDS) yang tersebar di berbagai daerah. Informasi mengenai DDS yang dicatat oleh pihak Bank Darah adalah informasi kode desa, golongan darah, dan volume darah yang dapat didonorkan oleh DDS. Contoh data tersebut adalah sebagai berikut.

Tabel 4.5 Contoh Data DDS Program Pengelolaan Bank Darah

Kode DDS	1	2	3	4	5	6	7	8
Kode Desa	3	1	3	1	2	3	1	2
Golongan Darah	A	B	A	AB	B	O	O	A
Volume Darah yang Dapat Didonorkan (ml)	150	250	300	450	200	350	500	500

Karena semakin lama, data DDS yang harus dikelola oleh Bank Darah tersebut semakin banyak, bahkan bisa jutaan orang, mereka meminta kamu untuk membangun sebuah program yang dapat menyimpan dan mengolah data DDS tersebut. Hasil pengolahan data tersebut akan digunakan oleh Bank Darah untuk menentukan pelaksanaan kegiatan Bakti Donor Darah yang lebih efektif dan tepat sasaran. Program tersebut harus bisa menentukan desa-desa yang harus dikunjungi untuk diadakan kegiatan bakti donor darah. Agar kamu merasakan proses pemrograman yang iteratif, kamu akan membuat program tersebut melalui empat tahap berikut:



Gambar 4.3 Empat Tahap Pembuatan Program Pengelolaan Bank Darah

Sumber: Faiz Alfian/Kemendikbudristek (2024)

Catatan: Untuk menghindari kesulitan dalam entri data golongan darah AB, *problem* ini akan memberikan kode golongan darah A = 1, B = 2, AB = 3, dan O = 4. kamu boleh saja mengkode dengan kode lainnya dengan syarat harus konsisten.

Subproblem 1: Menghitung Jumlah Pendonor Darah

Deskripsi:

Pada *subproblem* ini, kamu akan diminta untuk menghitung banyaknya pendonor darah di setiap desa berdasarkan kode desa yang diberikan.

Format Masukan:

Baris pertama adalah sebuah bilangan bulat N yang merupakan jumlah DDS. Nilai N ini paling banyak berjumlah 1000 orang. Baris kedua adalah M yang merupakan jumlah desa yang paling banyak berjumlah 10 desa. N baris berikutnya masing-masing akan terdiri atas tiga buah informasi, yaitu kode desa, golongan darah, dan volume darah yang dapat didonorkan. Kode desa merupakan sebuah bilangan bulat dari 1 sampai M , sedangkan volume darah yang dapat didonorkan adalah sebuah bilangan bulat dari 0 hingga 500 ml.

Format Keluaran:

Keluaran berupa banyaknya pendonor yang ada di masing-masing desa. Jumlah pendonor dicetak dengan format Desa M : Banyaknya Pendonor.

Tabel 4.6 Format Keluaran Program Pengelolaan Bank Darah
Subproblem 1

Contoh Masukan	Contoh Keluaran
8	Desa 1: 3
3	Desa 2: 2
3 1 150	Desa 3: 3
1 2 250	
3 1 300	
1 3 450	
2 2 200	
3 4 350	
1 4 500	
2 1 500	

Subproblem 2: Menghitung Jumlah Pendonor dan Total Volume Darah

Deskripsi:

Pada *subproblem* ini, kamu akan diminta untuk menghitung jumlah pendonor darah dan total volume darah yang dapat didonorkan di setiap desa berdasarkan kode desa dan golongan darah.

Format Masukan:

Sama dengan *subproblem* 1.

Format Keluaran:

Keluaran berupa jumlah pendonor dan total volume darah yang ada di masing-masing desa berdasarkan golongan darahnya. Ikuti format yang ada di contoh keluaran berikut:

Tabel 4.7 Format Keluaran Program Pengelolaan Bank Darah
Subproblem 2

Contoh Masukan	Contoh Keluaran
8	Desa 1:
3	• A : 0 0
3 1 150	• B : 1 250
1 2 250	• AB: 1 450
3 1 300	• O : 1 500
1 3 450	Desa 2:
2 2 200	• A : 1 500
3 4 350	• B : 1 200
1 4 500	• AB: 0 0
2 1 500	• O : 0 0
	Desa 3:
	• A : 2 450
	• B : 0 0
	• AB: 0 0
	• O : 1 350

Subproblem 3: Memilih Tempat Pelaksanaan Donor Darah

Deskripsi:

Pada *subproblem* ini, kamu akan diminta membantu Bank Darah dalam memilih tempat pelaksanaan Bakti Donor Darah ketika stok suatu golongan darah habis. Karena pelaksanaan bakti donor darah membutuhkan biaya, kamu harus meminimalkan jumlah desa tempat bakti donor darah akan dilaksanakan, dengan tetap mencapai jumlah volume darah yang dibutuhkan.

Format Masukan:

Apabila kebutuhan darah dapat dipenuhi, cetak daftar kode-kode desa yang akan dilaksanakan bakti golongan darah beserta jumlah darah yang dibutuhkan, dan satu kalimat “Kebutuhan darah dipenuhi dengan surplus X ml”. Sebaliknya, apabila kebutuhan darah tidak dapat dipenuhi, cetak “Kebutuhan darah masih kurang X ml” setelah seluruh keluaran dicetak.

Misalnya pada contoh kasus uji di *subproblem* 2, jika ditambahkan dengan 2 300 (artinya, ada kebutuhan 300ml golongan darah B) maka program haruslah menghasilkan keluaran berikut.

Desa 1 : 250 ml

Desa 2 : 200 ml

Kebutuhan darah dipenuhi dengan surplus 150 ml.

Dengan mengadakan bakti donor darah di kedua desa (Desa 1 dan Desa 2), kita dapat mengumpulkan total 450ml darah golongan B, sehingga menyisakan kelebihan 150ml. Tidak ada cara lain yang dapat dilakukan untuk memenuhi kebutuhan darah golongan B dengan mengadakan bakti donor darah di satu desa saja.

Subproblem 4: Tingkatkan Penyimpanan dan Kecepatan Olah Data DDS

Deskripsi:

Subproblem 4 sama dengan *Subproblem* 3. Hanya saja yang berbeda adalah ukuran *problem* yang diberikan. Program kamu sekarang harus bisa menyimpan dan mengolah data para Donor Darah Sukarela dari seluruh Indonesia, sehingga batasan masukan naik menjadi sebagai berikut.

Jumlah DDS : maksimal 30 juta orang

Jumlah Desa : maksimal 100.000 desa

Selain itu, karena pengambilan keputusan harus diambil secara cepat, program kamu harus dapat berjalan dalam waktu paling lama 1 detik. Buatlah program yang dapat melakukan hal tersebut.



Ayo Merancang Program



Merancang Algoritma Pengelolaan Donor Darah

Jenis Aktivitas: Berpasangan

No Aktivitas: LD-K11-03

Berdasarkan deskripsi permasalahan di atas, secara individu, definisikanlah permasalahan dan rancanglah algoritma solusi dari permasalahan tersebut. kamu dapat membuka kembali bahan belajar tentang pengelolaan donor darah yang menjadi domain permasalahan yang diberikan. Dokumentasikanlah setiap langkah yang kamu kerjakan, termasuk apa yang kamu hasilkan dalam Buku Kerja kamu.

Setelah kamu selesai merancang algoritma, secara berpasangan, saling tukarkan algoritma kamu. Setelah itu, telusurilah algoritma teman kamu dan cek apakah algoritma tersebut sudah benar atau belum. Apabila belum benar, secara bersama-sama, diskusikanlah apa yang dapat diperbaiki dari rancangan algoritma kamu. Jangan lupa untuk membandingkan solusi yang telah kamu hasilkan. Apabila solusi kamu berbeda, tapi sama-sama menghasilkan jawaban yang benar, bandingkanlah kedua solusi tersebut.



Ayo Membuat Program



Membuat Program Pengelolaan Donor Darah

Jenis Aktivitas: Berpasangan

No Aktivitas: LD-K11-04

Sekarang, secara individu, implementasikanlah algoritma yang telah kamu rancang dalam bentuk program dengan menggunakan bahasa pemrograman yang telah kamu kuasai. Sebelum program kamu kompilasi, secara berpasangan, saling tukarkan kode program kamu dan cek apakah kode program tersebut sudah ditulis dengan benar. Setelah itu, kompilasi kode tersebut menjadi program, dan ujlilah program teman kamu dengan

kasus uji yang kamu rancang. Apabila program teman kamu belum menghasilkan jawaban yang benar, sampaikanlah kepada teman kamu agar ia dapat memperbaiki kode program tersebut hingga menghasilkan jawaban yang benar. Setelah selesai, presentasikanlah hasil kerja kamu di depan kelas dengan mengikuti petunjuk dari guru.

Ini Berpikir Komputasional!

Kegiatan yang telah kamu kerjakan di atas adalah penerapan pemrograman untuk menyelesaikan permasalahan yang ada di dunia nyata. Terlihat bahwa informatika dapat digunakan untuk mengambil keputusan yang lebih baik berdasarkan data akurat yang diolah dengan cepat. Tentunya, *problem* yang diberikan di atas adalah penyederhanaan dari *problem* yang ada di lapangan, yang memiliki kompleksitas yang lebih tinggi.

Pada saat membaca deskripsi *subproblem* keempat, kamu mungkin kaget melihat jumlah data yang sangat besar. Program yang telah kamu buat untuk mengerjakan *Subproblem* 3 belum tentu dapat mengerjakan *Subproblem* 4 dalam waktu maksimal 1 detik yang diberikan. Yang berbeda dari *Subproblem* 3 dengan *Subproblem* 4 adalah ukuran dari *problem* yang lebih besar. Di sinilah salah satu tantangan dari *big data*, yaitu bagaimana kita dapat merancang algoritma yang paling efisien sehingga data yang besar tadi tetap dapat diolah dalam waktu yang singkat.

3. Problem Persilangan Tanaman

Pada bagian ini, kamu akan membuat program untuk mensimulasikan persilangan tanaman menggunakan Hukum Mendel yang telah kamu pelajari pada mata pelajaran Biologi. *Problem* akan dibagi menjadi beberapa *subproblem* dengan tingkat kesulitan yang meningkat. Ikutilah petunjuk guru kamu dalam memilih tingkat kesulitan *subproblem* yang akan kamu kerjakan. Apabila kamu berhasil mengerjakan *subproblem* tersebut, kamu dapat menantang diri kamu untuk mengerjakan *subproblem* yang lebih sulit.

Setelah ditentukan tingkat kesulitan *subproblem* yang diberikan, kamu dapat mengerjakan aktivitas Ayo Merancang Program: Merancang Algoritma

Persilangan Tanaman dan Ayo Buat Program: Membuat Program Persilangan Tanaman berdasarkan deskripsi permasalahan yang sesuai.

Problem: Persilangan Benih Padi Unggulan

Tania, seorang peneliti di bidang pemuliaan tanaman, sedang melakukan persilangan tanaman padi untuk menghasilkan benih unggulan. Sifat tanaman padi yang akan ia perhatikan adalah tinggi tanaman, jumlah anakan, jumlah bulir, dan ketahanan terhadap penyakit. Seluruh sifat ini adalah dominan, dengan kata lain gen tanaman yang tinggi mendominasi tanaman yang rendah. Untuk membantunya melakukan persilangan, Tania membuat sebuah program yang dapat membantunya menghitung rasio peluang munculnya tanaman dengan karakteristik tertentu apabila kedua “orang tua”-nya diketahui sifatnya.

Oleh karena Tania baru bisa membuat program yang membaca bilangan, Tania harus membuat kode sendiri agar ia dapat memasukkan karakteristik tanaman padi yang akan ia silangkan. Ia membuat aturan pemberian kode seperti berikut.

Pertama, Tania memberikan kode 0 sampai 2 untuk mewakili genotipe dari masing-masing fenotipe. Misalnya pada fenotipe Tinggi Tanaman, Tania memberikan nilai 2 untuk genotipe TT, nilai 1 untuk genotipe Tt, dan nilai 0 untuk genotipe tt.

Tabel 4.8 Kode Sifat Fenotipe dan Genotipe Tanaman Tania

Sifat	Fenotipe	Kode	Sifat	Fenotipe	Kode
Tinggi Tanaman	Tinggi (TT)	2	Jumlah Bulir	Banyak (BB)	2
	Tinggi (Tt)	1		Banyak (Bb)	1
	Pendek (tt)	0		Sedikit (bb)	0
Jumlah Anakan	Banyak (AA)	2	Ketahanan terhadap Penyakit	Resisten (RR)	2
	Banyak (Aa)	1		Resisten (Rr)	1
	Sedikit (aa)	0		Rentan (rr)	0

Kedua, Tania akan menuliskan masukan program berupa satu atau lebih nilai 0, 1, atau 2 yang merupakan sifat dari tanaman yang akan ia silangkan. Angka di posisi pertama merupakan tinggi tanaman, posisi kedua merupakan jumlah anakan, posisi ketiga merupakan jumlah bulir, dan posisi keempat

adalah ketahanan terhadap penyakit. Sebagai contoh, jika Tania akan memasukkan tanaman yang tinggi (TT), jumlah anakan sedikit (aa), jumlah bulir sedikit (bb), dan resisten terhadap penyakit (Rr), maka bilangan yang akan Tania masukkan adalah: 2 0 0 1. Sekarang, kamu dapat mulai membantu Tania membuat program. Selamat mengerjakan!

Subproblem 1: Menghitung Persentase Suatu Sifat Keturunan Pertama (F1)

Deskripsi:

Pada *subproblem* ini, Tania hanya memperhatikan sifat Tinggi Tanaman dan Jumlah Anakan, dan ingin menghitung persentase dari suatu keturunan generasi pertama (F1).

Format Masukan:

Masukan berupa tiga baris. Dua baris pertama adalah sifat dari kedua orang tua, dan baris ketiga adalah sifat dari keturunan pertama yang akan dihitung persentase kemunculan fenotipenya.

Format Keluaran:

Keluaran berupa sebuah persentase fenotipe dari tanaman di baris ketiga yang ditulis sebagai bilangan pecahan dengan dua tempat desimal dan diakhiri dengan tanda %.

Tabel 4.9 Format Keluaran Program Persilangan Tanaman *Subproblem 1*

Contoh Masukan	Contoh Keluaran
2 2 0 0 0 0	25.00%

Penjelasan:

Pada contoh tersebut, Tania menyilangkan dua induk, yang bersifat tinggi (TT) dan banyak anakan (AA) dengan pendek (tt) dan sedikit anakan (aa). Lalu, Tania ingin menghitung persentase generasi pertama dengan sifat pendek (tt) dan sedikit anakan (aa).

Rasio fenotipe dari persilangan ini adalah Tinggi dan Banyak Anakan dengan Pendek dan Sedikit Anakan = 3 : 1 = 75% : 25%.

Subproblem 2: Mengetahui Semua Sifat Keturunan Pertama (F1)

Deskripsi:

Pada *subproblem* ini, Tania hanya memperhatikan sifat Tinggi Tanaman dan Jumlah Anakan, dan ingin mengetahui sifat dari semua keturunan pertama (F1).

Format Masukan:

Masukan berupa dua baris yang berisi sifat dari kedua orang tua.

Format Keluaran:

Keluaran berupa 16 baris, masing masing baris berisi sifat dari semua keturunan pertama.

Tabel 4.10 Format Keluaran Program Persilangan Tanaman
Subproblem 2

Contoh Masukan	Contoh Keluaran
2 2	Tinggi Banyak
1 1	Tinggi Banyak
	Tinggi Banyak
	Tinggi Banyak
	Tinggi Banyak
	Tinggi Sedikit
	Tinggi Banyak
	Tinggi Sedikit
	Tinggi Banyak
	Tinggi Banyak
	Pendek Banyak
	Pendek Banyak
	Tinggi Banyak
	Tinggi Sedikit
	Pendek Banyak
	Pendek Sedikit

Penjelasan:

Pada contoh tersebut, Tania menyilangkan dua induk, yang bersifat tinggi (TT) dan banyak anakan (AA) dengan pendek (tt) dan sedikit anakan (aa). Lalu,

Tania ingin menghitung persentase generasi kedua dengan sifat pendek (tt) dan sedikit anakan (aa). Rasio fenotipe dari persilangan ini adalah sebagai berikut.

Tinggi dan Banyak Anakan : Pendek dan Sedikit Anakan = 15 : 1 = 93.75% : 6.25%.

Subproblem 3: Menghitung Persentase Suatu Sifat Keturunan Kedua (F2)

Deskripsi:

Pada *subproblem* ini, Tania akan memperhatikan keempat sifat tanaman padi tersebut, dan ingin menghitung persentase dari suatu keturunan generasi kedua (F2).

Format Masukan:

Masukan berupa tiga baris. Dua baris pertama adalah sifat dari kedua orang tua, dan baris ketiga adalah sifat dari keturunan kedua yang akan dihitung persentase kemunculan fenotipnya.

Format Keluaran:

Keluaran berupa sebuah persentase fenotipe dari tanaman di baris ketiga yang ditulis sebagai bilangan pecahan dengan dua tempat desimal dan diakhiri dengan tanda %.

Tabel 4.11 Format Keluaran Program Persilangan Tanaman
Subproblem 3

Contoh Masukan	Contoh Keluaran
2 2 2 2 0 0 0 0 0 0 0 0	0.39%

Penjelasan:

Pada contoh tersebut, Tania menyilangkan dua induk yang bersifat seperti berikut.

- 1: Tinggi (TT), Banyak Anakan (AA), Banyak Bulir (BB), dan Resisten (RR)
- 2: Pendek (tt), Sedikit Anakan (aa), Sedikit Bulir (bb), dan Rentan (rr)

Lalu, Tania ingin menghitung persentase generasi kedua dengan sifat:

Pendek (tt), Sedikit Anakan (aa), Sedikit Bulir (bb), dan Rentan (rr). Persentase fenotipe generasi kedua dari sifat tersebut adalah sama dengan 0.390625% yang jika dibulatkan ke atas menjadi 0.39%.



Ayo Merancang Program



Merancang Algoritma Persilangan Tanaman

Jenis Aktivitas: Berpasangan

No Aktivitas: LD-K11-05

Berdasarkan deskripsi permasalahan di atas, secara individu, definisikanlah permasalahan dan rancanglah algoritma solusi dari permasalahan tersebut. Kamu dapat membuka kembali bahan belajar tentang persilangan tanaman yang menjadi domain permasalahan yang diberikan. Dokumentasikanlah setiap langkah yang kamu kerjakan, termasuk apa yang kamu hasilkan dalam Buku Kerja kamu.

Setelah kamu selesai merancang algoritma, secara berpasangan, tukarkan algoritma kamu. Setelah itu, telusurilah algoritma teman kamu dan cek apakah algoritma tersebut sudah benar atau belum. Apabila belum benar, secara bersama-sama, diskusikanlah apa yang dapat diperbaiki dari rancangan algoritma kamu. Jangan lupa untuk membandingkan solusi yang telah kamu hasilkan. Apabila solusi kamu berbeda, tapi sama-sama menghasilkan jawaban yang benar, bandingkanlah kedua solusi tersebut.



Ayo Membuat Program



Membuat Program Persilangan Tanaman

Jenis Aktivitas: Berpasangan

No Aktivitas: LD-K11-06

Sekarang, secara individu, implementasikanlah algoritma yang telah kamu rancang dalam bentuk program dengan menggunakan bahasa pemrograman yang telah kamu kuasai. Sebelum program kamu kompilasi, secara berpasangan, tukarkan kode program kamu dan cek apakah kode program tersebut sudah ditulis dengan benar. Setelah itu, kompilasi kode tersebut menjadi program, dan ujilah program teman kamu dengan kasus uji yang kamu rancang. Apabila program teman kamu belum menghasilkan jawaban yang benar, sampaikanlah kepada teman kamu agar ia dapat memperbaiki kode program tersebut hingga menghasilkan jawaban yang benar. Setelah selesai, presentasikanlah hasil kerja kamu di depan kelas, mengikuti petunjuk dari guru.

Ini Berpikir Komputasional!

Pada kelas X, kamu telah belajar bahwa informatika dapat diterapkan di berbagai bidang keilmuan. Aktivitas di atas adalah salah satu contoh penerapan informatika di bidang biologi. Pada saat mengerjakan aktivitas di atas, kamu memanfaatkan pengetahuan kamu di pemrograman dan juga biologi untuk menyelesaikan permasalahan yang diberikan. Pada aktivitas ini, juga kamu merasakan pengalaman bekerja berpasangan dalam membuat program. Membuat program yang kompleks seringkali melibatkan suatu tim yang harus bekerja sama dan saling melengkapi satu sama lain. Yang kamu lakukan di aktivitas ini adalah bagaimana kamu dapat saling mengecek hasil pekerjaan untuk menghasilkan program yang benar dan berkualitas tinggi di dalam tim. Pada aktivitas ini, kamu juga telah merasakan bagaimana berpikir komputasional diterapkan dalam pemrograman. Kamu dapat mencoba mengidentifikasi prinsip berpikir komputasional apa saja yang kamu terapkan dalam menyelesaikan *problem* tersebut.

4. Problem Stoikiometri

Stoikiometri adalah ilmu yang mempelajari dan menghitung hubungan kuantitatif dari reaktan dan produk dalam reaksi kimia. Pada bagian ini kamu akan diajak untuk membangun sebuah program yang dapat membantu kamu memproses masukan berupa suatu persamaan reaksi kimia. Manusia mengobservasi pada reaksi kimia pada alam. Dengan menggunakan berbagai hukum seperti hukum kekekalan massa, hukum perbandingan tetap, dan hukum perbandingan berganda, manusia melahirkan suatu model reaksi kimia. Salah satunya adalah stoikiometri.

Setelah memahami stoikiometri, kamu dapat membaca sebuah persamaan reaksi kimia dan mengetahui bahwa persamaan tersebut benar karena semua pereaksi dikonsumsi (habis bereaksi), tidak ada kekurangan pereaksi, dan tidak ada kelebihan pereaksi (sisanya). Sebelum masuk ke *problem* yang akan dikerjakan, kamu dapat mencoba menuliskan struktur proses yang kamu lakukan untuk mengecek persamaan reaksi berikut apakah benar atau tidak.



Problem: Simulasi Stoikiometri

Tujuan kamu dalam *problem* ini adalah membuat program yang dapat mengecek benar tidaknya persamaan reaksi kimia yang diberikan. Program yang dapat mengecek seluruh kemungkinan persamaan kimia sangat kompleks. Oleh karena itu, kita memerlukan batasan-batasan yang dapat menyederhanakan masalah tersebut, yaitu sebagai berikut.

- Penulisan unsur kimia pada masukan sudah sesuai dengan aturan. Misal: Li untuk litium dan H untuk hidrogen.
- Persamaan reaksi yang diproses tidak melibatkan senyawa ionik dan senyawa yang memiliki tanda kurung.
- Persamaan reaksi yang diproses dijamin benar.
- Selain itu, aturan penulisan yang menggunakan *International Union of Pure and Applied Chemistry* (IUPAC) seperti pada persamaan reaksi kimia berikut perlu disederhanakan.



akan kita tulis dalam *problem* ini sebagai berikut:

1 Fe 2 O 3

2 Al 1

>

1 Al 2 O 3

1 Fe 2

Jika kamu tertarik, kamu bisa saja membuat program yang dapat membaca masukan berupa persamaan reaksi kimia seperti yang biasa dituliskan dengan mempelajari konsep *parsing*. *Parsing* adalah sebuah proses menganalisis string yang mengikuti aturan formal tertentu sehingga string tersebut dapat dibaca oleh program. Dalam hal ini, aturan formal yang diikuti adalah aturan formal penulisan reaksi kimia. *Parsing* tidak dibahas pada buku ini, tetapi kamu dapat mencari tahu lebih lanjut tentang konsep *parsing* dari berbagai sumber lainnya.

Subproblem 1: Menghitung Atom

Deskripsi:

Pada *subproblem* ini, kamu harus membuat program untuk menghitung banyaknya atom unsur tertentu yang terdapat pada senyawa kimia yang diberikan. Program kamu akan membaca sebuah senyawa dengan format yang telah ditentukan di bagian sebelumnya, kemudian mencetak banyaknya atom dari masing-masing unsur penyusun senyawa tersebut.

Format Masukan:

Senyawa yang telah diformat sesuai dengan deskripsi *problem*.

Format Keluaran:

Untuk setiap unsur, cetak lambang dari unsur tersebut dan banyaknya atom yang menyusun unsur tersebut. Kedua nilai ini dipisahkan oleh satu karakter spasi dan diakhiri dengan *newline* atau baris baru berikutnya.. Lambang unsur dan banyaknya atom dicetak berurutan mulai dari unsur paling kiri pada penulisan senyawa.

Tabel 4.12 Format Keluaran Program Stoikiometri *Subproblem 1*

Contoh Masukan	Contoh Keluaran
1 C 1 H 3 Cl 1	C 1 H 3 Cl 1
2 H 2 O 1	H 4 O 2

Penjelasan:

Senyawa pada contoh pertama adalah CH_3Cl dan contoh kedua adalah $2\text{H}_2\text{O}$.

Subproblem 2: Mengecek Persamaan Reaksi

Deskripsi:

Pada *subproblem* ini, kamu akan membuat program yang akan membaca sebuah persamaan reaksi kimia dan menentukan apakah persamaan reaksi kimia tersebut setimbang. Suatu atom dijamin hanya akan muncul 1 kali pada reaktan dan 1 kali pada produk. Apabila setimbang, maka program cukup

mengeluarkan pesan “Reaksi Setimbang”. Apabila persamaan tersebut tidak setimbang, maka program akan mengeluarkan pesan “Reaksi Tidak Setimbang” dan menyajikan banyak atom dari reaktan yang kurang atau berlebih.

Format Masukan:

Sebuah persamaan reaksi kimia dengan format yang telah dijelaskan pada bagian penjelasan *problem*. Untuk mengakhiri masukan, kamu akan menginput sebuah karakter “F”.

Format Keluaran:

Jika persamaan setimbang, cetak “Reaksi Setimbang”. Jika reaksi tidak setimbang, program akan mencetak lambang dari unsur yang kurang atau berlebih beserta dengan selisih jumlah atomnya. Unsur dicetak berurutan mulai dari unsur yang paling kiri. Untuk setiap unsur tersebut, cetak dalam satu baris dengan format Lambang Unsur: jumlah atom. Jumlah atom diberi tanda negatif jika kurang dan diberi tanda positif jika berlebih.

Contoh:

Misalnya program menerima masukan berupa persamaan reaksi kimia sebagai berikut.



Program akan melihat bahwa persamaan tersebut tidak setimbang dan akan mengeluarkan keluaran berikut:

Tabel 4.13 Format Keluaran Program Stoikiometri *Subproblem 2*

Contoh Masukan	Contoh Keluaran
2 Fe 2 O 3 2 Al 1 > 2 Al 2 O 3 2 Fe 1 F	Reaksi Tidak Setimbang Fe:+2 Al:-2

Subproblem 3: Menghitung Hasil Reaksi

Deskripsi:

Pada *subproblem* ini, program akan menerima masukan berupa persamaan reaksi kimia. Bagian reaktan dari persamaan tersebut sudah ditulis dengan koefisien yang lengkap. Namun, koefisien dari bagian produk persamaan tersebut belum lengkap. Suatu atom dijamin hanya akan muncul 1 kali pada reaktan dan 1 kali pada produk. Setelah membaca persamaan tersebut, program akan mencetak koefisien dari setiap molekul di bagian produk.

Format Masukan:

Sebuah persamaan reaksi kimia dengan format yang telah dijelaskan pada bagian penjelasan *problem*. Namun, koefisien pada seluruh molekul produk diberi nilai 0. Untuk mengakhiri masukan, kamu akan menginput sebuah karakter “F”.

Format Keluaran:

Keluaran dari program adalah koefisien dari setiap molekul produk yang dihasilkan dari persamaan tersebut. Koefisien tersebut harus membuat persamaan reaksi yang setimbang. Jika bagian produk terdiri atas lebih dari satu molekul, koefisien dicetak dari molekul produk yang paling kiri dan dipisahkan dengan karakter spasi.

Contoh:

Misalnya program menerima masukan berupa persamaan reaksi kimia sebagai berikut.



Program akan melihat bahwa persamaan tersebut tidak setimbang dan akan mengeluarkan keluaran berikut:

Tabel 4.14 Format Keluaran Program Stoikiometri *Subproblem 3*

Contoh Masukan	Contoh Keluaran
2 Fe 2 O 3 4 Al 1 > 0 Al 2 O 3 0 Fe 1 F	2 4

Dengan koefisien 2 dan 4 pada sisi produk, maka persamaan reaksi kimia akan setimbang menjadi:



Subproblem 4: Melengkapi Koefisien Reaktan dan Produk

Deskripsi:

Pada *subproblem* ini, persamaan reaksi kimia yang dibaca seluruhnya memiliki koefisien 0. Program harus membaca persamaan reaksi kimia tersebut dan mencetak koefisien setiap molekul sedemikian sehingga reaksi setimbang. Karena kemungkinan koefisien ini sangat banyak, kamu hanya perlu mencetak kemungkinan koefisien yang paling kecil. Terdapat tepat 2 senyawa pada reaktan dan 2 senyawa pada hasil dan masing-masing atom hanya muncul tepat 1 kali pada reaktan dan 1 kali pada hasil.

Format Masukan:

Sebuah persamaan reaksi kimia dengan format yang telah dijelaskan pada bagian penjelasan *problem*, tetapi koefisien pada seluruh molekul reaktan dan produk diberi nilai 0.

Format Keluaran:

Keluaran program adalah persamaan reaksi kimia yang telah dilengkapi dengan koefisien terkecil yang membuat persamaan reaksi tersebut setimbang.

Contoh:

Misalnya program menerima masukan berupa persamaan reaksi kimia sebagai berikut.



Program akan mencetak persamaan reaksi berikut:

Tabel 4.15 Format Keluaran Program Stoikiometri *Subproblem 4*

Contoh Masukan	
1	Fe 2 O 3
2	Al 1
>	
1	Al 2 O 3
2	Fe 1



Ayo Merancang Program



Q Merancang Algoritma Simulasi Stoikiometri

Jenis Aktivitas: Berpasangan

No Aktivitas: LD-K11-07

Berdasarkan deskripsi permasalahan di atas, secara individu, definisikanlah permasalahan dan rancanglah algoritma solusi dari permasalahan tersebut. Kamu dapat membuka kembali bahan belajar tentang stoikiometri yang menjadi domain permasalahan yang diberikan. Dokumentasikanlah setiap langkah yang kamu kerjakan, termasuk apa yang kamu hasilkan dalam Buku Kerja kamu.

Setelah kamu selesai merancang algoritma, tukarkan algoritma yang kamu buat kepada pasanganmu. Setelah itu, telusurilah algoritma teman kamu dan cek apakah algoritma tersebut sudah benar atau belum. Apabila belum benar, secara bersama-sama, diskusikanlah apa yang dapat diperbaiki dari rancangan algoritma kamu. Jangan lupa untuk membandingkan solusi yang telah kamu hasilkan. Apabila solusi kamu berbeda, tapi sama-sama menghasilkan jawaban yang benar, bandingkanlah kedua solusi tersebut.



Ayo Membuat Program



Q Membuat Program Simulasi Stoikiometri

Jenis Aktivitas: Berpasangan

No Aktivitas: LD-K11-08

Sekarang, secara individu, implementasikanlah algoritma yang telah kamu rancang dalam bentuk program dengan menggunakan bahasa pemrograman yang telah kamu kuasai. Sebelum program kamu kompilasi, tukarkan kode program dengan milik pasanganmu dan cek apakah kode program tersebut sudah ditulis dengan benar. Setelah itu, kompilasi kode tersebut menjadi program, dan ujilah program teman kamu dengan kasus uji yang kamu rancang. Apabila program teman kamu belum menghasilkan jawaban yang benar, sampaikanlah kepada teman kamu agar ia dapat memperbaiki kode program tersebut hingga menghasilkan jawaban yang benar. Setelah selesai, presentasikanlah hasil kerja kamu di depan kelas, mengikuti petunjuk dari guru.

Ini Berpikir Komputasional!

Pada *problem* stoikiometri ini, selain kamu membuat program yang menyimulasikan suatu aturan di bidang kimia, kamu pun belajar membuat program yang menerima masukan yang dekat dengan bahasa yang digunakan manusia.

B. Pengembangan Aplikasi *Mobile* dengan App Inventor

Saat ini kehidupan sehari-hari manusia banyak dibantu oleh aplikasi atau perangkat lunak yang terpasang pada ponsel pintar, komputer, atau diakses secara daring lewat peramban. Aplikasi tersebut di antaranya adalah aplikasi perkantoran, aplikasi bertukar pesan, pemutar lagu, aplikasi desain, dan pengolah akuntansi. Aplikasi dapat dibedakan berdasarkan platform pengembangan dan penggunaannya, yaitu aplikasi desktop, aplikasi web, dan aplikasi *mobile*. Penulisan aplikasi sering disingkat menjadi *apps*.

1. Desktop Apps

Aplikasi desktop adalah aplikasi yang dikembangkan dengan tujuan implementasi pada komputer desktop atau piranti lokal komputer. Aplikasi ini harus dipasang pada piranti lokal komputer. Setelah terpasang aplikasi ini akan berada pada memori dari piranti lokal.

2. Web Apps

Aplikasi berbasis web adalah aplikasi yang dikembangkan dengan tujuan dapat diakses menggunakan koneksi jaringan komputer dan internet menggunakan protokol *http*. Aplikasi ini tidak terpasang pada piranti atau komputer desktop lokal, tetapi terpasang pada server tertentu. Aplikasi ini kebanyakan diakses menggunakan *browser*, namun ada juga yang berbentuk *client side* di mana ada program kecil yang terpasang pada piranti lokal, tetapi proses komputasi utama dilakukan di server.

3. Mobile Apps

Aplikasi *mobile* yang juga disebut dengan *mobile apps* adalah aplikasi yang dirancang untuk dapat dieksekusi pada piranti *mobile* seperti ponsel, *tablet*, atau *smart watch*. *Mobile* memiliki arti mudah bergerak. Aplikasi *mobile* dapat dipasang pada ponsel, tablet, atau gadget lainnya. Aplikasi ini berkembang

pesat karena kemudahan penggunaan piranti *mobile* dan dapat diintegrasikan dengan sistem lain yang ada pada piranti *mobile* seperti *GPS*, kamera, dan sidik jari. Saat ini tersedia jutaan aplikasi *mobile* yang ada di platform pendistribusian aplikasi digital yaitu Play Store ataupun Apps Store.

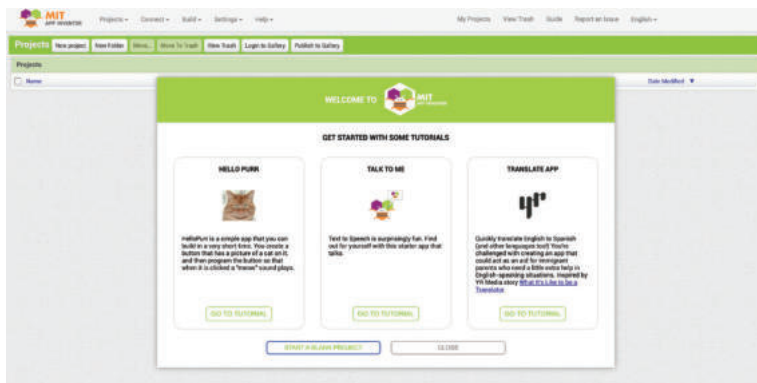
Pengembangan aplikasi banyak dibantu oleh perkakas pengembangan yang disebut dengan *Integrated Development Environment* (IDE). IDE membantu kemudahan dan efektifitas pengembangan perangkat lunak. Salah satu IDE yang digunakan untuk pengembangan aplikasi *mobile* adalah App Inventor.

4. App Inventor

App Inventor adalah perangkat lunak IDE terintegrasi yang berbasis web. App Inventor pada awalnya dikembangkan oleh Google yang saat ini dipelihara oleh Massachusetts Institute of Technology (MIT). App Inventor memungkinkan pemrogram komputer pemula dapat membuat *mobile apps* di atas OS Android maupun iOS. Aplikasi App Inventor bersifat *open source* dan *free*.

App Inventor memiliki antarmuka berbasis grafis dan memiliki tampilan yang mirip dengan bahasa pemrograman block Scratch/Blockly yang telah kamu pelajari di kelas 7 dan 8. Dengan App Inventor kamu dapat membuat program dengan cara seret lepas (*drag and drop*) komponen-komponennya. App Inventor sebagai perkakas, terus dikembangkan kecanggihannya melalui riset intensif di bidang *educational computing*. App Inventor mendukung penggunaan *cloud data* dengan Firebase dan Firebase Realtime Database.

App Inventor dapat diakses dengan peramban melalui tautan <https://ai2.appinventor.mit.edu>. Tampilan awal App Inventor tampak pada gambar berikut:



Gambar 4.4 Tampilan Awal App Inventor

Sumber: Faiz Alfian/Kemendikbudristek (2024)

App Inventor memiliki banyak komponen yang dapat digunakan dalam pembuatan aplikasi. Komponen tersebut dikelompokkan menjadi *User Interface Components*, *Layout Components*, *Media Components*, *Drawing and Animation Components*, dan lain-lain. *User Interface Components* memiliki komponen-komponen yang berhubungan dengan antarmuka pengguna, seperti: *Button* (Tombol), *CheckBox*, *DatePicker*, *Image*, dan lain-lain. Masing-masing komponen memiliki *methods*, *events*, dan *properties* yang digunakan untuk memanipulasi komponen tersebut.

Properties pada komponen adalah atribut yang mendeskripsikan sifat dari komponen, misalnya lebar tombol, warna dari teks, dll. *Properties* biasanya dapat dibaca dan di-set, tetapi ada juga *properties* yang hanya bisa dibaca.

Methods adalah fungsi yang dapat dikenakan pada komponen yang memilikinya, *methods* dapat digunakan untuk mengatur *properties*.

Events adalah kejadian yang terjadi karena pemanggilan *methods*, seperti aksi *mouse click* yang menghasilkan *mouse events* yang menyebabkan suatu fungsi/prosedur akan dieksekusi.

Sebagai contoh, komponen *Button* (Tombol) yang memiliki kemampuan untuk mendeteksi penekanan tombol memiliki *properties*: warna background tombol yang dapat diubah sesuai keinginan, tombol dapat di-set *enabled* (aktif) atau tidak aktif, dan font yang dapat diatur menjadi *italic*, **bold**, dan lain-lain. *Properties* tersebut dapat dimanipulasi menggunakan Designer Editor atau pada Blocks Editor. Tabel berikut memberikan berbagai contoh komponen, *methods*, *events*, dan *properties* dari komponen pada App Inventor.

Tabel 4.16 Contoh Komponen App Inventor

Components	Properties	Events	Methods
<i>User Interface Components</i>			
Button (Tombol): Memiliki kemampuan mendeteksi penekanan (klik) dari user	<i>BackgroundColor(Color)</i>	<i>Click()</i>	(tidak ada)
	<i>Enabled(Boolean)</i>	<i>GotFocus()</i>	
	<i>FontSize(Number)</i>	<i>LongClick()</i>	
	<i>FontBold(Boolean)</i>	<i>LostFocus()</i>	
	<i>FontItalic(Boolean)</i>	<i>TouchDown()</i>	
	<i>Image(Text)</i>		

Components	Properties	Events	Methods
TextBox: Kotak tempat user mengisi teks	<i>Text(Text)</i>	<i>GotFocus()</i>	<i>HideKeyboard()</i>
	<i>BackgroundColor(Color)</i>	<i>LostFocus()</i>	<i>RequestFocus()</i>
	<i>Enabled(Boolean)</i>		
	<i>FontSize(Number)</i>		
	<i>FontBold(Boolean)</i>		
	<i>FontItalic(Boolean)</i>		
	<i>Multiline(Boolean)</i>		
Label: Komponen untuk menampilkan teks	<i>Text(Text)</i>	<i>(tidak ada)</i>	<i>(tidak ada)</i>
	<i>BackgroundColor(Color)</i>		
	<i>Enabled(Boolean)</i>		
	<i>FontSize(Number)</i>		
	<i>FontBold(Boolean)</i>		
	<i>FontItalic(Boolean)</i>		
	<i>TextColor(Color)</i>		
Layout Components			
Horizontal Arrangement: Komponen ini memformat elemen yang diletakkan pada layar akan tertampil secara horizontal dari kiri ke kanan	<i>AlignHorizontal(Number)</i>	<i>(tidak ada)</i>	<i>(tidak ada)</i>
	<i>Align Vertical (Number)</i>		
	<i>BackgroundColor(Color)</i>		
	<i>Image(Text)</i>		
Media Components			
TextToSpeech: Komponen yang akan mengubah teks menjadi suara	<i>Language(Text)</i>		
	<i>Country(Text)</i>		
	<i>SpeechRate(Number)</i>		



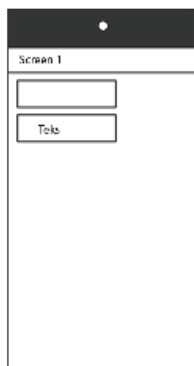
Starter App Inventor - Halo Dunia! dengan *Text to Speech*

Jenis Aktivitas: Kelompok

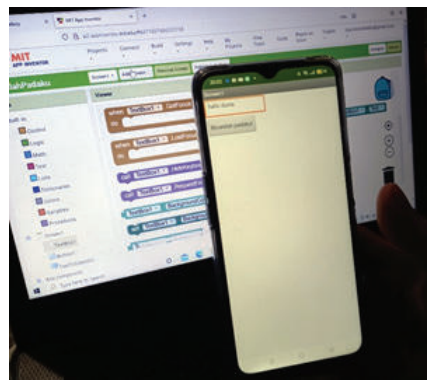
No Aktivitas: LD-K11-09

1. Siapkan komputer yang terkoneksi dengan internet dan terpasang perangkat lunak MIT AI2 Companion atau ponsel atau tablet dengan sistem operasi Android/iOS. Namun jika ponsel tidak tersedia maka kamu dapat menggunakan emulator ponsel yang akan muncul pada layar komputer kamu. Kamu harus melakukan pengaturan khusus untuk emulator ini. Pastikan kamu telah memahami pemrograman dengan Scratch/Blockly yang dipelajari di SMP.
2. Kamu akan mengembangkan aplikasi *mobile* yang memiliki antarmuka sebagai berikut:

Sketsa



Hasil Akhir



Sumber: Dela Chaerani/Kemendikbudristek (2024)

3. Spesifikasi Aplikasi:
 - a. *Input*: Pengguna mengetikkan “Halo Dunia” pada *textbox* dan mengetuk tombol di bawah *textbox*
 - b. *Proses*: Aplikasi mengubah teks yang ditulis program menjadi suara
 - c. *Output*: Aplikasi akan memperdengarkan suara speaker ponsel
4. Persiapan aplikasi
 - d. Masuk atau *login* ke situs App Inventor (<https://appinventor.mit.edu/>) dengan menggunakan akun Google kamu.

**Jika belum memiliki akun Google, kamu dapat menggunakan milik orang tua, kakak, atau meminta bantuan gurumu.*

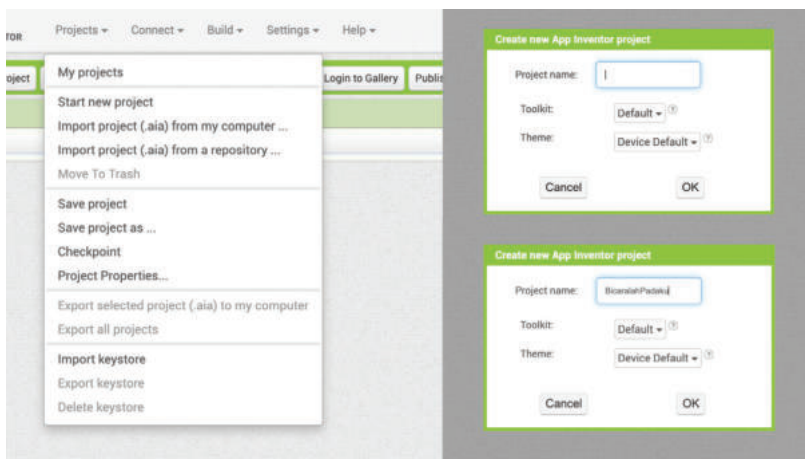
- e. Klik *Continue* saat layar pembuka (*splash screen*) muncul.



Sumber: Sashibhusan_Sahoo/community.appinventor.mit.edu (2021)

5. Pengkodean

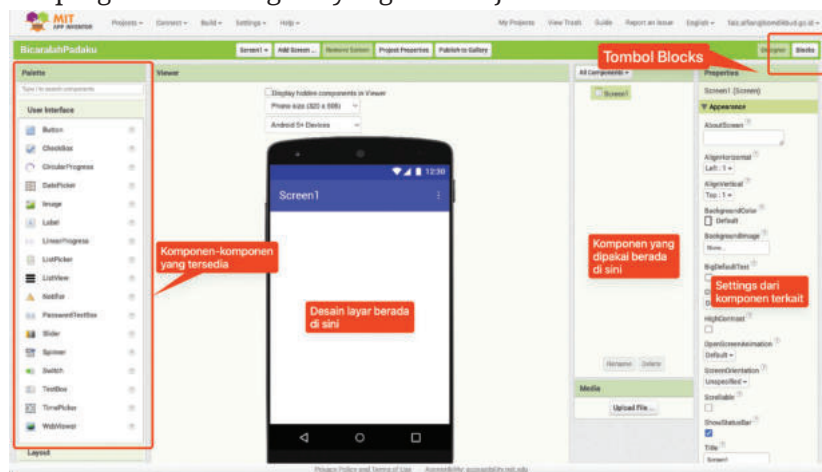
- a. Buat proyek baru dengan memilih menu *Projects* lalu klik *Start new project*. Lalu akan muncul *menu pop-up* untuk mengisi nama proyek, kamu beri nama proyek baru tersebut dengan “Bicaralah KEPADAKU” (tanpa spasi). Perlu diketahui setiap kamu membuat aplikasi di App Inventor, aplikasi tersebut disimpan dalam sebuah proyek yang berisi semua *file* terkompilasi ke dalam sebuah *executable file*. *File* tersebut dapat berisi kode sumber, ikon, gambar, suara, *file* data, dan sebagainya.



Sumber: Faiz Alfian/Kemendikbudristek (2024)

6. Perancangan *User Interface* (UI)

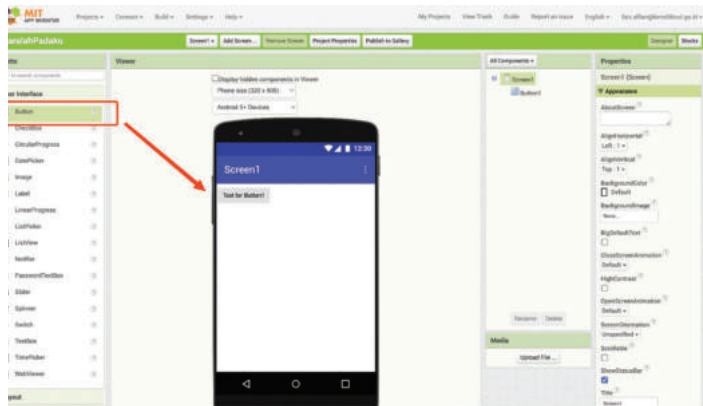
- Pengembangan aplikasi mobile dilakukan dengan merancang *User Interface* (UI) dan perancangan blok kode.
- Perancangan UI dilakukan dengan menggunakan tampilan *Designer*, yang tampil dengan mengklik tombol *Designer* pada bagian kanan atas. Tampilan *Designer App Inventor* memiliki empat kolom. Kolom *Palette* merupakan tempat komponen-komponen yang tersedia dari App Inventor, kolom *Viewer* merupakan kolom untuk perancangan UI aplikasi, kolom *Components* berisi komponen-komponen yang digunakan pada proyek, dan kolom *Properties* yang merupakan kolom untuk melakukan pengaturan terhadap komponen-komponen yang digunakan.
- Perancangan blok kode dilakukan dari Tampilan *Blocks* yang tampil dengan mengklik tombol *Blocks* pada bagian kanan atas di samping tombol *Designer* yang akan dijelaskan kemudian.



Sumber: Faiz Alfian/Kemendikbudristek (2024)

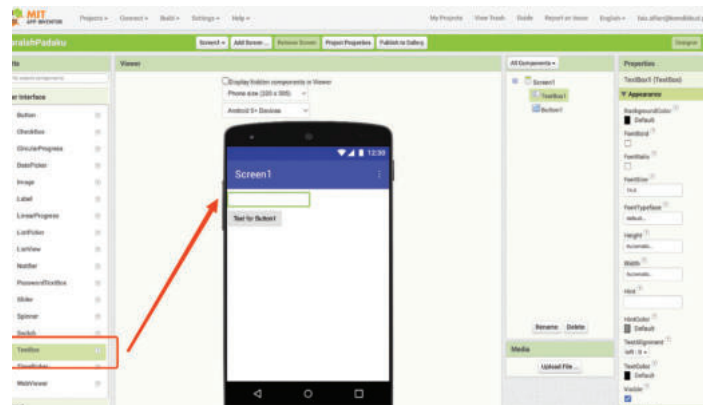
7. Mengaplikasikan perancangan UI:

- Tambahkan Tombol *Button* pada *Viewer*, dengan seret dan lepaskan (*drag and drop*) dari kolom *Palette* ke kolom *Viewer*. Sebuah *Button* dengan nama *Text for Button1* akan tercipta. Nama tersebut dapat diganti dengan nama lain yang sesuai.



Sumber: Faiz Alfian/Kemendikbudristek (2024)

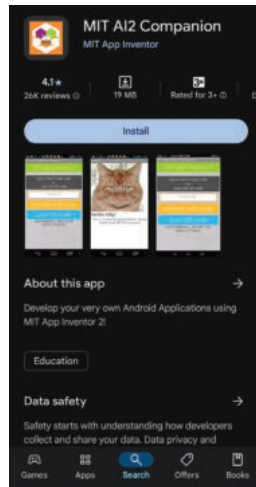
- b. Tambahkan *TextBox* dengan *drag and drop* ke area *Viewer*. Secara otomatis *TextBox* dengan nama default *TextBox1* akan muncul.



Sumber: Faiz Alfian/Kemendikbudristek (2024)

8. Persiapan Pengujian

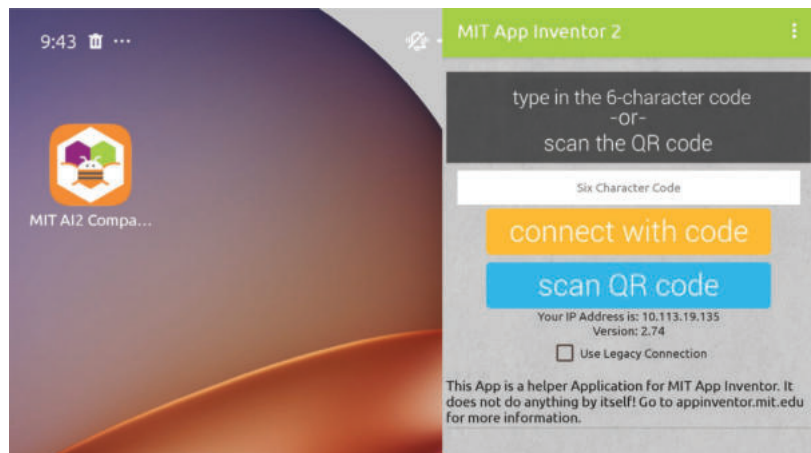
- a. Aplikasi yang dikembangkan adalah aplikasi *mobile* yang berjalan pada piranti *mobile phone*, sehingga pengujian idealnya dilakukan dengan menguji dengan ponsel secara *live*, langkah persiapan pengujian dilakukan dengan langkah berikut.
 - i Sambungkan App Inventor pada komputer ke ponselmu dengan kabel USB atau perangkat Wi-Fi.
 - ii Unduh dan pasang/*install* MIT AI2 Companion di PlayStore/ App Store pada ponsel kamu.



Sumber: Faiz Alfian/Kemendikbudristek (2024)

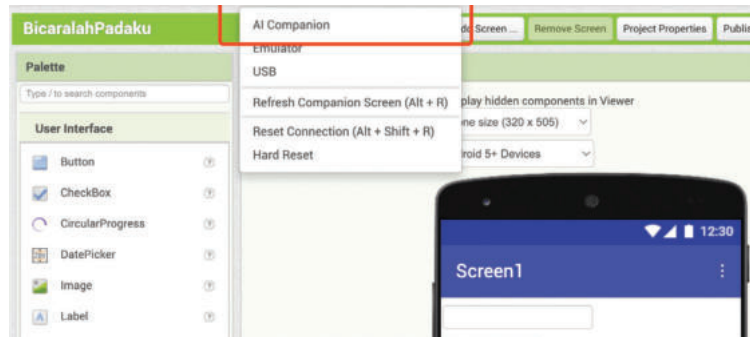
- iii Pengunduhan dan pemasangan AI Companion memerlukan pengaturan (*settings*) pada ponsel. Lakukan centang (*check*) untuk membolehkan “*Unknown Sources*” pada menu “*Security*” pada ponsel. Selanjutnya, Pindai Kode QR untuk mengunduh langsung MIT AI2 Companion atau klik “*Need help finding the Companion App?*”. Setelah selesai diunduh, pasang/*install* aplikasi MIT AI2 Companion.

Berikut tampilan App Inventor jika berhasil dipasang pada ponsel.



Sumber: Faiz Alfian/Kemendikbudristek (2024)

- iv Berikut ini tampilan App Inventor pada komputer/laptop. Untuk menghubungkan aplikasi App Inventor pada ponsel dengan App Inventor yang digunakan pada komputer/laptop, klik menu *Connect* pada App Inventor komputer/laptop kamu, lalu klik menu AI Companion.



Sumber: Faiz Alfian/Kemendikbudristek (2024)

Berikut ini adalah tampilan menu *pop-up* AI Companion komputer/laptop. Kode 6 digit dan kode QR akan muncul yang dapat digunakan untuk menghubungkan dua perangkat. Kamu dapat memasukkan kode 6 digit ke ponsel atau memindai kode QR dengan ponsel untuk menghubungkan kedua perangkat tersebut.

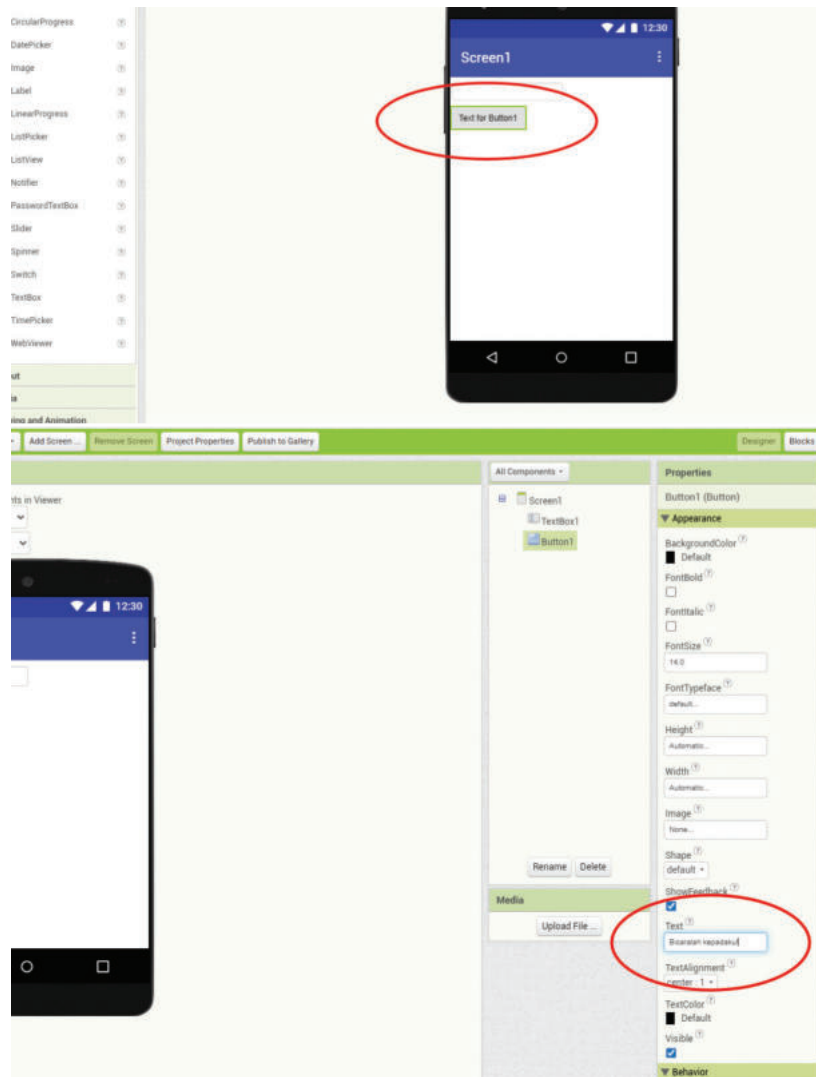


Sumber: Faiz Alfian/Kemendikbudristek (2024)

Selanjutnya akan muncul *progress bar* dalam komputer/ laptop kamu sebagai tanda proses menghubungkan kedua perangkat. Setelah itu, jika instalasi sukses, kamu dapat melihat *app* kamu di ponsel. Jika kamu menambahkan komponen lain pada *app* kamu di komputer, maka perubahan akan terjadi juga di ponsel. Selamat, ponsel kamu siap digunakan untuk pengujian aplikasi.

9. Lanjutan Pengkodean:

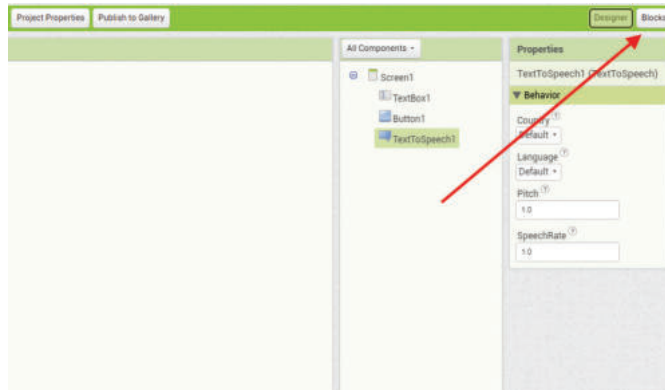
- a. Masih pada *Designer View*, ubahlah teks teks "Text for Button1", menjadi "Bicaralah kepadaku!" pada kolom *properties*.



Sumber: Faiz Alfian/Kemendikbudristek (2024)

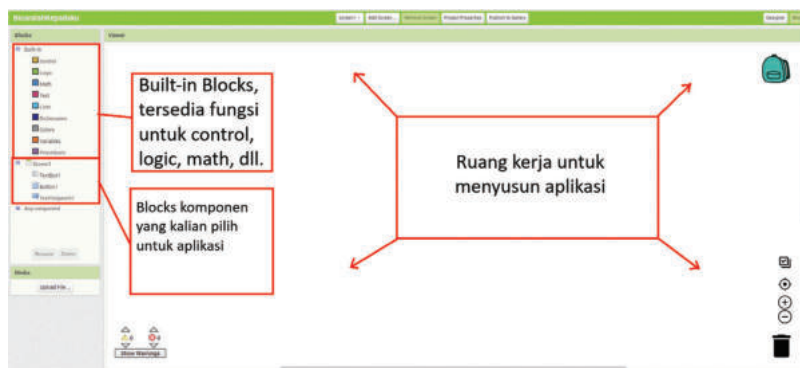
- b. Tambahkan komponen *TextToSpeech* dengan cara *drag and drop* komponen tersebut ke kotak *Viewer*. Pilih komponen dari menu *Pallette > Media > TextToSpeech*.
- c. Pengkodean Blok: Setelah komponen *TextToSpeech* ditambahkan, selanjutnya kamu lanjutkan pengkodean blok dengan masuk ke

mode editor *Blocks*. Beralih ke mode *editor Blocks* dilakukan dengan menekan tombol *Blocks* pada pojok kanan halaman.



Sumber: Faiz Alfian/Kemendikbudristek (2024)

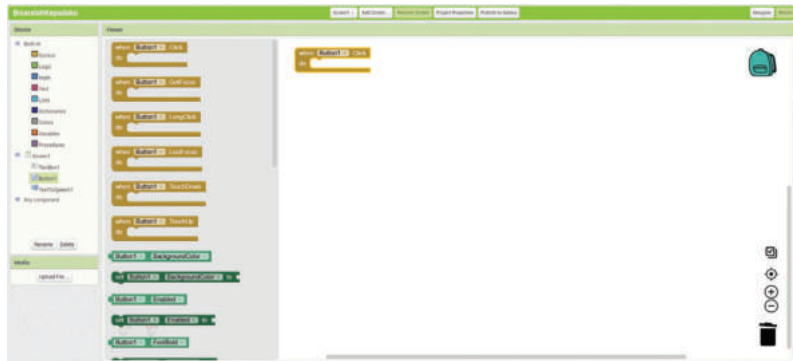
Editor Blocks adalah tempat untuk menyusun program dari aplikasi. Pada editor ini terdapat blok *Built-in* yang telah tersedia dan dapat digunakan untuk menangani operasi *Control*, *Logic*, *Math*, dll. *Blocks* merupakan ruang kerja yang digunakan untuk menyusun program aplikasi.



Sumber: Faiz Alfian/Kemendikbudristek (2024)

Pengkodean Blok selanjutnya dilakukan dengan langkah berikut ini:

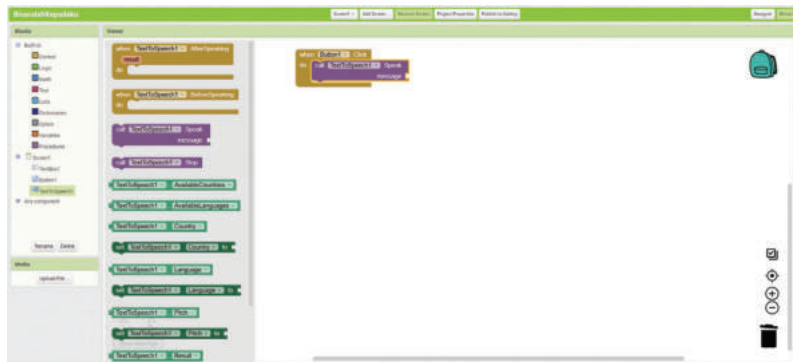
- i. Buatlah event ketika tombol `Button1` ditekan, dengan cara:
 - Klik tombol `Button1` pada kolom *Blocks*
 - Pilih blok `when Button1.Click` pada *Viewer*
 - *Drag and drop* pada *Viewer* yang kosong, yang hasilnya tampak seperti pada gambar berikut:



Sumber: Faiz Alfian/Kemendikbudristek (2024)

ii Tambahkan blok `call TextToSpeech1.Speak` dari komponen `TextToSpeech1` ke blok `when Button1.Click` dengan cara:

- Klik `TextToSpeech1` pada kolom *Blocks*
- Pilih blok `call TextToSpeech1.Speak` di kolom *Viewer*
- Drag and drop blok `call TextToSpeech1.Speak` pada kolom *Viewer* yang kosong, yang hasilnya tampak seperti pada gambar berikut:



Sumber: Faiz Alfian/Kemendikbudristek (2024)

iii Tambahkan blok `TextBox1.Text` dari komponen `TextBox1` ke blok `call TextToSpeech1.Speak` dengan cara:

- Klik `TextBox1` pada kolom *Blocks*
- Pilih blok `call TextBox1.Text` di kolom *Viewer*
- Drag and drop blok `TextBox1.Text` ke blok `call TextToSpeech1.Speak` pada kolom *Viewer*, yang hasilnya tampak seperti pada gambar berikut:



Sumber: Faiz Alfian/Kemendikbudristek (2024)

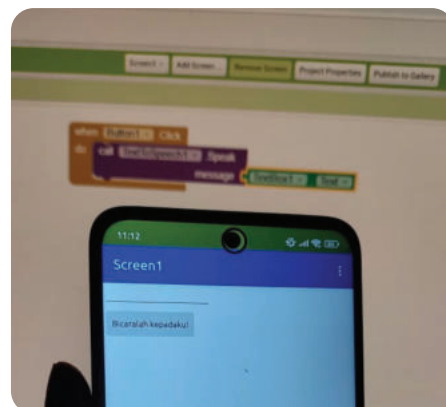
- iv Simpan *file* proyek kamu dengan memilih menu *Projects > Save Project*

10. Pengujian Aplikasi:

Pengujian aplikasi dilakukan dengan menuliskan teks **“Halo Dunia”** pada `TextBox1`, dan mengetuk tombol **“Bicaralah kepadaku!”** pada ponsel yang telah tersambung dengan komputer/laptop sebelumnya pada langkah 3. Jika speaker pada ponsel mengeluarkan suara Halo Dunia maka kamu telah berhasil membuat aplikasi mobile pertama kamu. kamu saat ini menggunakan *TextToSpeech*, yang merupakan library App Inventor yang berfungsi mengubah teks menjadi suara, meniru kamu (manusia) membaca teks dan mengucapkannya! kamu tinggal memakai, dan tidak perlu tahu betapa rumitnya program didalamnya.

Setelah itu cobalah mengetikkan teks yang berbeda, atau dengan bahasa yang berbeda, misalnya bahasa Inggris atau Perancis, dan tekan tombol Bicaralah Padaku. Apakah yang terjadi?

Berikut ini foto dari **aplikasi BicaralahKepadaku** yang telah selesai dibuat:



Sumber: Faiz Alfian/
Kemendikbudristek (2024)

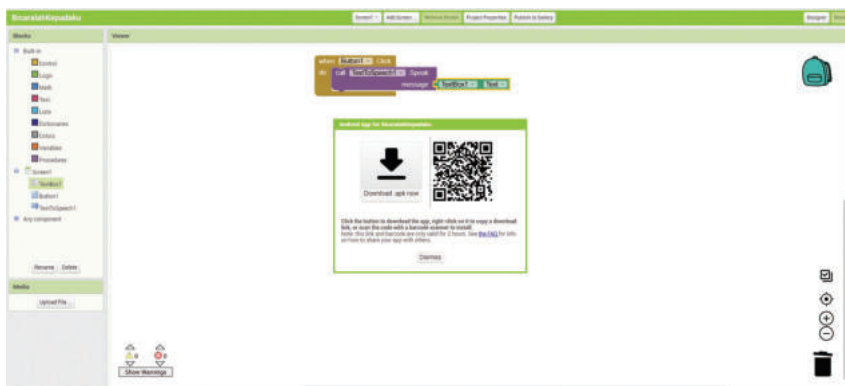
Aplikasi yang telah kamu kembangkan dapat diunduh dan kamu bagikan ke teman dan orang tua, dengan cara:

- a. Pilih menu *Build* dan pilih Android App (.apk).



Sumber: Faiz Alfian/Kemendikbudristek (2024)

- b. Klik *Download .apk now*, dan file .apk akan terunduh. File .apk adalah file paket android (*Android Application Package*) yang digunakan untuk mendistribusikan aplikasi, file dapat digandakan dan dipasang pada piranti *mobile* kamu dan teman-teman kamu.



Sumber: Faiz Alfian/Kemendikbudristek (2024)



Speechboard

Jenis Aktivitas: Individu

No Aktivitas: LD-K11-10

Pada aktivitas ini kamu akan belajar untuk mengembangkan aplikasi yang mampu memainkan sebuah rekaman pidato dengan menyentuh sebuah gambar.

Kebutuhan Alat dan Bahan:

1. Komputer yang terkoneksi dengan internet, ponsel/tablet dengan sistem operasi Android atau iOS. Komputer juga harus terpasang perangkat lunak MIT AI2 Companion. Jika ponsel tidak tersedia dapat digunakan *emulator* pada komputer.
2. Gambar proklamator pada saat mengucapkan proklamasi kemerdekaan Indonesia (*file type .jpg*) dan rekaman pidato pembacaan teks proklamasi pada tanggal 17 Agustus 1945 (*file type .mp3*). *File* dapat diunduh di https://static.buku.kemdikbud.go.id/content/media/rar/Informatika_XI.rar

Prasyarat:

1. Kamu harus telah memahami materi pemrograman menggunakan Scratch/Blockly yang dipelajari pada jenjang SMP.
2. Kamu memahami cara mengunduh *file* gambar dan suara dari internet, dan menyimpannya di komputer.

Deskripsi Produk :

Kamu akan mengembangkan aplikasi mobile yang memiliki antarmuka sebagai berikut.



Sumber: Dela Chaerani/Kemendikbudristek (2024)

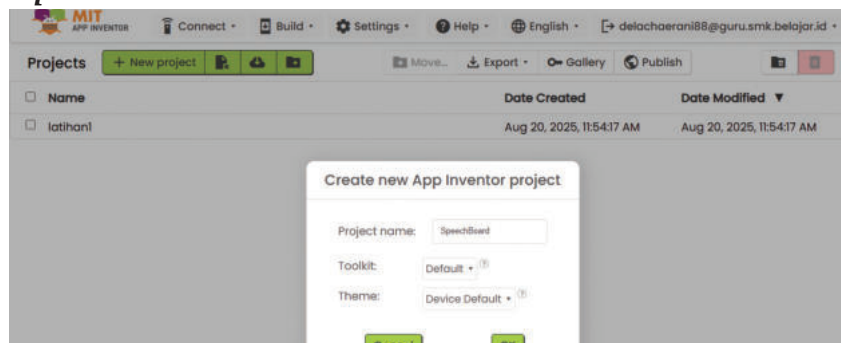
Spesifikasi Aplikasi:

- *Input*: Pengguna mengetuk gambar Sang Proklamator Indonesia di ponsel
- *Proses*: Aplikasi Memainkan rekaman pidato proklamasi kemerdekaan Indonesia
- *Output*: Aplikasi memperdengarkan suara rekaman lewat speaker ponsel

Langkah-langkah:

1. Persiapan:

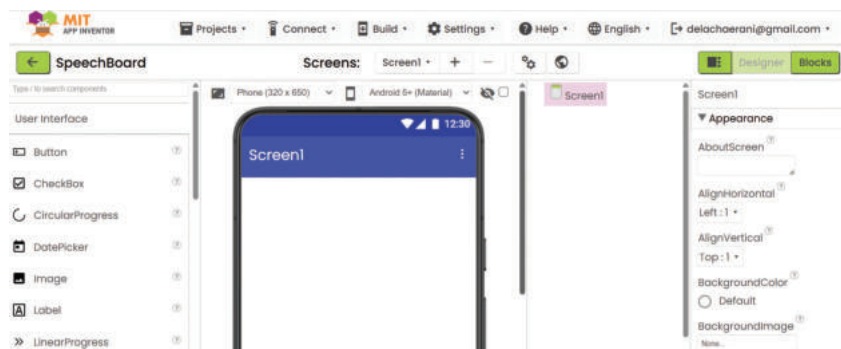
- a. Buka aplikasi App Inventor dengan mengakses <https://ai2.appinventor.mit.edu>.
- b. Mulailah membuat proyek baru, namailah proyek dengan nama **“SpeechBoard”**.



Sumber: Dela Chaerani/Kemendikbudristek (2024)

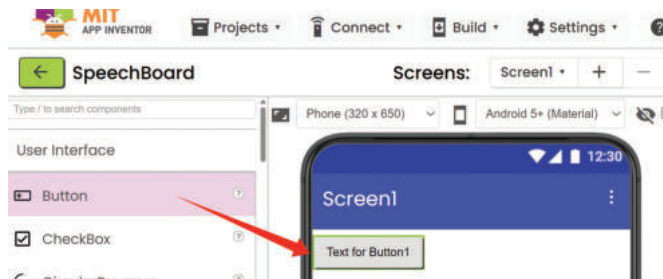
2. Perancangan *User Interface* (UI):

- a. Berikut tampilan awal dari proyek,



Sumber: Dela Chaerani/Kemendikbudristek (2024)

- b. Tambahkan Button pada layar dengan cara seret dan lepaskan (*drag and drop*)



Sumber: Dela Chaerani/Kemendikbudristek (2024)

- c. Ubahlah background Button pada screen dengan gambar Proklamasi Kemerdekaan RI, dengan sebelumnya mengunggah gambarnya (*file .png*).
- d. Tambahkan dua label dengan teks “Proklamasi Kemerdekaan Republik Indonesia” dan "Klik pada gambar untuk memulai suara" pada bagian atas dan bawah *button*. "Ubah properti 'Label1' seperti tampilan berikut (tampilan properti ada di kanan layar). Seperti contoh berikut:



Sumber: Dela Chaerani/Kemendikbudristek (2024)

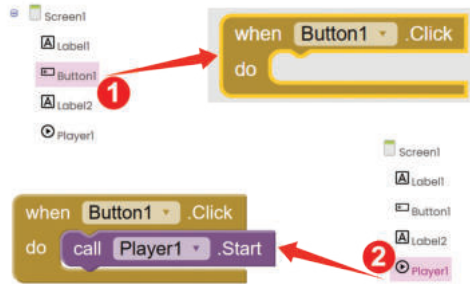
- e. Tambahkan komponen *Player* dari kolom *Palette > Media*, sehingga *Player1* tercipta, dan *upload file* pidato kemerdekaan (*file .mp3*) sebagai *source* dari *Player1*. "Ubah menu properti-source '*Player1*' dengan file suara.



Sumber: Dela Chaerani/Kemendikbudristek (2024)

3. Pengkodean Blok:

- Berikutnya, kamu harus menambahkan blok kode dengan beralih ke *mode* Blocks.
- Tambahkan kode program untuk memainkan file pidato dengan blok *when* Button1.Click.
- Isi blok *call* Player1.Start ke dalam blok *when* Button1.Click



Sumber: Dela Chaerani/Kemendikbudristek (2024)

- Dan terakhir jangan lupa untuk menyimpan proyek (*save project*) kamu.

4. Pengujian:

Ujilah kode program dengan mengetuk tombol pada ponsel untuk memainkan rekaman pidato, seperti pada langkah pengujian aktivitas LD-K11-09. Jika program telah berhasil memperdengarkan suara proklamasi, maka program kamu telah sesuai dengan spesifikasi, dan lanjutkan dengan aktivitas pengembangan.



Ayo Kembangkan



Modifikasi Aplikasi

Jenis Aktivitas: Kelompok

No Aktivitas: LD-K11-11

Deskripsi Proyek:

Kamu diharapkan mengembangkan proyek perangkat lunak berbasis *mobile* yang dinamakan *speechboard*, dengan melakukan modifikasi aplikasi tersebut dengan menambahkan beberapa pidato dari para pahlawan Indonesia, misalnya Ki Hajar Dewantara, Sutomo, dll. kamu akan mengembangkan aplikasi *mobile* yang memiliki antarmuka sebagai berikut:



Spesifikasi Aplikasi:

- **Input:** Pengguna mengetuk salah satu gambar pahlawan Indonesia di ponsel
- **Proses:** Aplikasi memainkan rekaman pidato yang terkenal dari para pahlawan tersebut
- **Output:** Aplikasi akan memperdengarkan rekaman pidato dari pahlawan yang dipilih lewat speaker ponsel

Gunakan Lembar Kegiatan Peserta Didik saat kamu mengembangkan aplikasi untuk berbagi peran dan tugas, berikut:

LKPD-01

Format Lembar Kegiatan Peserta Didik Pengembangan Aplikasi

Peran	Penanggung Jawab
Analisis Program:	
a. Deskripsi Produk	
b. Spesifikasi Aplikasi	
c. Kebutuhan resource: <i>file</i> , alat, dll	
Perancang <i>User Interface</i> (UI)	
Pemrogram Kode	
Penguji Program	

Spesifikasi (Deskripsi Produk, Fungsionalitas Aplikasi, Kebutuhan Resource)

Rancangan User Interface (UI)

Kode Program

Pengujian

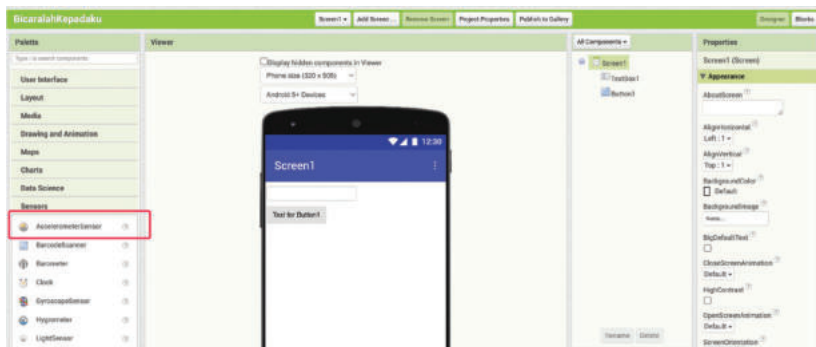
Diisi saat perencanaan			Diisi setelah pengujian		
No	Fitur	Dikerjakan Oleh	Sesuai dengan spesifikasi		Keterangan Hasil Pengujian
			Ya	Tidak	

Selanjutnya, kembangkanlah aplikasi untuk dapat menghentikan suara (*pause*) ketika gambar diketuk, dan memperdengarkan suara kembali ketika diketuk lagi (*toggle*). Setelah Itu,kembangkan juga dengan hanya boleh satu *player* yang hidup pada suatu saat tertentu. Jangan lupa setelah ditambah fungsionalitasnya ujilah kembali aplikasi kamu tersebut.

5. Pengayaan

Untuk meningkatkan pemahaman dan kemampuan kamu dalam materi Pengembangan Aplikasi *Mobile* ini, kamu dapat mengikuti aktivitas pengayaan berikut. Aplikasi yang telah kamu buat pada aktivitas LD-K11-09 dan LD-K11-10 dapat dikembangkan menjadi aplikasi lain, misalnya:

- Aplikasi pada aktivitas LD-K11-10 dapat dikembangkan dengan memperdengarkan suara teks saat ponsel digerakkan naik turun. Aplikasi ini akan mengakses *sensor accelerometer*, sehingga teks akan disuarakan ketika ponsel digerakkan naik turun.



Gambar 4.5 Letak Menu Sensor Accelerometer

Sumber: Dela Chaerani/Kemendikbudristek (2024)

- Mengembangkan aplikasi pemanggil nama peserta didik, sehingga guru dapat memanggil peserta didik dengan menekan tombol pada aplikasi di ponsel.

C. Pengembangan Aplikasi Kecerdasan Buatan dengan App Inventor

Tahukah kamu saat ini teknologi Kecerdasan Buatan telah banyak diimplementasikan pada kehidupan kita sehari-hari. kamu mungkin pernah mendengar **Google Assistant**, **Apple Siri**, **Amazon Alexa** yang merupakan aplikasi asisten pribadi yang dapat melakukan pekerjaan tertentu dengan perintah menggunakan suara. Saat ini banyak perusahaan di Indonesia yang menggunakan *chatbot* untuk berinteraksi dengan konsumen secara otomatis, atau ketika kamu menggunakan youtube maka akan muncul video rekomendasi yang sesuai dengan kesukaan kamu.

Nama-nama produk diatas adalah contoh-contoh produk hasil dari Kecerdasan Buatan, dan masih banyak contoh lain yang digunakan di industri dalam bentuk robot otomasi industri, robot penjelajah ruang angkasa, dll.

1. Kecerdasan Buatan (*Artificial Intelligence*)

Kecerdasan buatan atau *Artificial Intelligence* (AI) adalah kecerdasan yang dimiliki oleh sistem atau mesin atau komputer. AI mampu untuk melakukan tugas yang umumnya terkait dengan kemampuan makhluk cerdas. Istilah ini sering diterapkan pada proyek pengembangan sistem yang memiliki sifat intelektualitas manusia, seperti kemampuan untuk menalar, menemukan makna, melakukan generalisasi, atau belajar dari pengalaman masa lalu. Sejak perkembangan komputer digital pada tahun 1940-an, AI telah banyak diimplementasikan untuk melakukan tugas yang kompleks seperti, misalnya, menemukan bukti untuk teorema matematika atau bermain catur dengan sangat mahir.

Namun, meskipun kemajuan terus-menerus dalam kecepatan pemrosesan komputer dan kapasitas memori, belum ada program yang dapat menandingi fleksibilitas manusia dalam domain yang lebih luas atau dalam tugas-tugas yang membutuhkan banyak pengetahuan sehari-hari. Di sisi lain, beberapa program telah mencapai tingkat kinerja yang sangat impresif yang dapat menggantikan para ahli dan profesional manusia dalam melakukan tugas-tugas tertentu tertentu, seperti diagnosis medis, mesin pencari komputer, dan pengenalan suara atau tulisan tangan.

Kecerdasan Buatan kemudian berkembang dengan memunculkan berbagai subbidang yaitu:

a. *Machine Learning*

Machine Learning adalah mesin pembelajar yang mampu melakukan pembuatan model analitik secara otomatis. Mesin ini menggunakan beberapa metode berbasis statistik, jaringan saraf, fisika, dll untuk menemukan *insight* (wawasan) tersembunyi dari data tanpa diprogram secara eksplisit. Mesin ini mampu mengambil kesimpulan secara otomatis.

b. *Deep Learning*

Deep Learning adalah mesin pembelajar dengan pembelajaran mendalam menggunakan jaringan syaraf tiruan dengan ukuran dan lapisan unit pemrosesan yang besar. *Deep Learning* memanfaatkan kemajuan dalam

kemampuan komputasi dari perangkat komputer yang semakin cepat dan *algoritma training* (pelatihan) yang terus meningkat kinerjanya untuk mengenali pola kompleks dalam data besar. Aplikasi umum *Deep Learning* yang banyak digunakan adalah pengenalan gambar dan suara. Pada beberapa literatur disebutkan bahwa *Deep Learning* adalah subset dari *Machine Learning*, dan *Machine Learning* adalah subset dari Kecerdasan Buatan. Gambar berikut menunjukkan ilustrasi keterkaitan Kecerdasan Buatan, *Machine Learning*, dan *Deep Learning*.



Gambar 4.6 Kecerdasan Buatan (*Artificial Intelligence*)

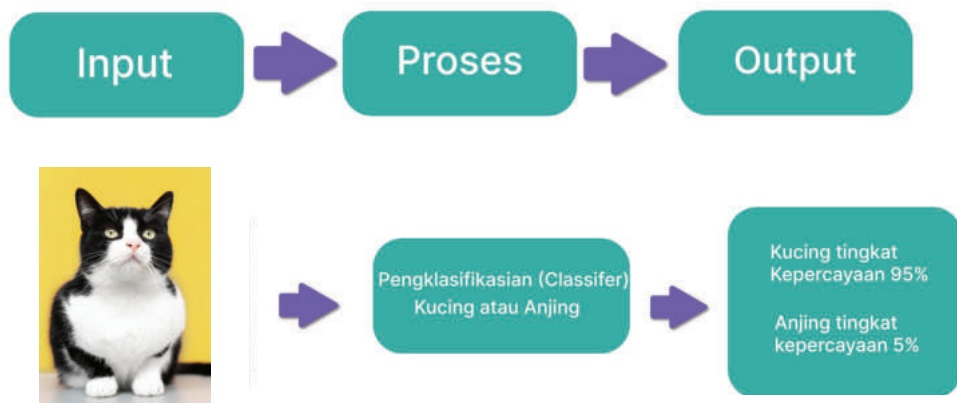
Sumber: Riksa Arif/Kemendikbudristek (2024)

Machine Learning adalah bagian dari bidang ilmu Kecerdasan Buatan yang mempelajari cara membuat mesin atau sistem yang memiliki kecerdasan dan menyerupai manusia yang mampu belajar dan memecahkan masalah. Beberapa contoh dari *machine learning* adalah *search engine* pada peramban, (contoh: Google, Bing), sosial media yang memiliki kemampuan memberi saran kepada pengguna yang biasa disebut recommendation system (contoh: Youtube, Netflix, *E-Commerce*), mobil otonom (contoh: Tesla), *game* dengan pengambilan keputusan otomatis (contoh: *strategic game*). *Machine learning* diharapkan menjadi sistem yang mampu belajar terus menerus. Dengan semakin banyak data yang dipelajari, sistem akan menjadi semakin pintar.

2. Cara Kerja Machine Learning untuk Klasifikasi Gambar (*Image*)

Klasifikasi gambar adalah salah satu fitur penting pada *machine learning*, sebagai contoh pada mobil otonom yang dapat bergerak tanpa sopir, mobil ini harus mampu dengan cepat untuk menginterpretasikan dan mengklasifikasi sebuah objek yang dilihat dari kamera. Mobil harus menentukan apakah objek

tersebut kendaraan lain, pejalan kaki (*pedestrian*) atau rambu lalu lintas. Hal ini sangat penting bagi mobil otonom karena sangat berpengaruh pada gerak jalan mobil. Salah satu *library/extension* perkakas *machine learning* di MIT App Inventor yang mampu mengklasifikasikan gambar adalah LookExtension. *Library* ini dapat menerima gambar yang diambil dari kamera sebagai input dan mampu mengklasifikasi gambar *input* tersebut menjadi output yang disajikan dalam bentuk teks/tulisan. Contoh pada gambar 4.6 adalah klasifikasi sebuah gambar apakah kelas/kategorinya adalah kucing atau anjing.

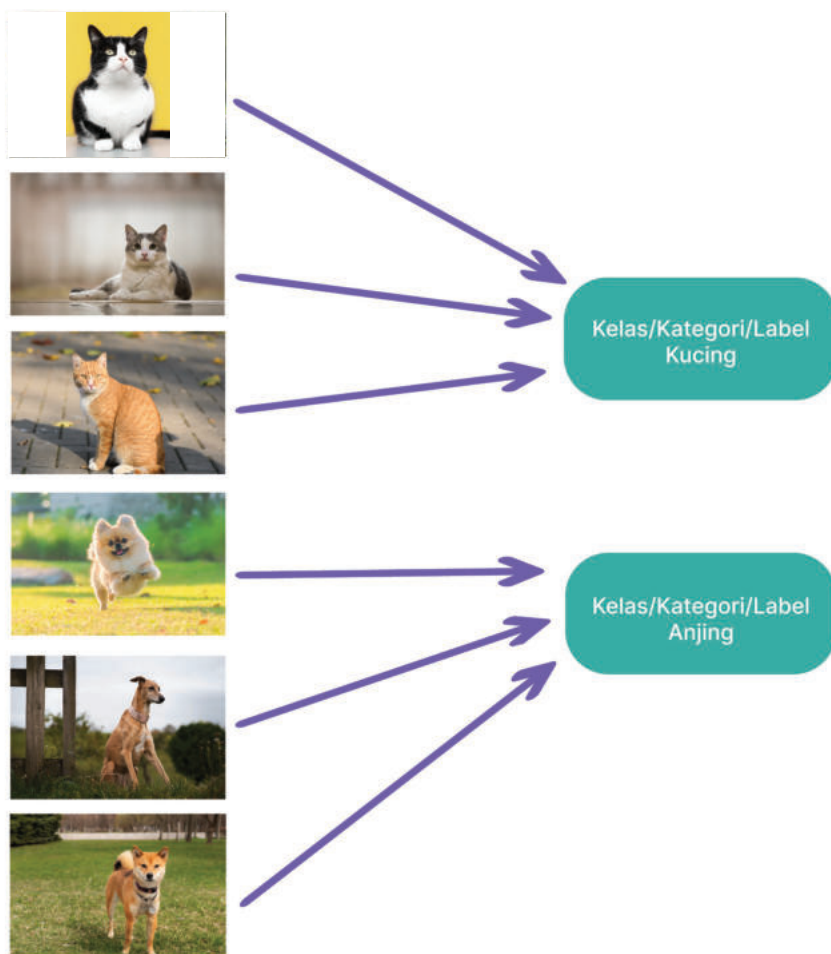


Gambar 4.7 Proses Klasifikasi Gambar

Sumber: freepik/Freepik

Untuk mendapatkan output klasifikasi berupa teks “Kucing (tingkat kepercayaan 95%)” atau “Anjing (tingkat kepercayaan 5%)” terjadi proses penghitungan/komputasi yang cukup tinggi dan kompleks. Proses komputasi akan menghasilkan kesimpulan sebuah gambar adalah kucing atau anjing.

Agar dapat menghasilkan kesimpulan yang mampu mengklasifikasikan gambar, sistem pada awalnya dilatih untuk dapat mengklasifikasikan gambar (*image*) tertentu. Sebagai contoh, jika kamu ingin mengembangkan sistem untuk dapat mengenali dan mengklasifikasi gambar kucing atau anjing, maka kamu harus melatih sistem dengan memberikan banyak gambar kucing dan memberi nama kategori (*label*) kucing dan memberikan banyak gambar anjing serta memberi label atau kategori anjing juga agar sistem dapat mengingat dan mengenalinya. Memberikan kategori atau label sangat penting bagi sistem, sehingga ketika sistem diberikan input gambar baru, sistem akan dapat menentukan apakah gambar tersebut lebih mirip gambar kucing atau anjing.



Gambar 4.8 Gambar Kucing dan Anjing serta Kelasnya

Sumber: freepik/Freepik

Dengan contoh gambar kucing dan anjing yang cukup, program akan terlatih untuk menentukan/mengklasifikasi suatu gambar adalah anjing atau kucing. Secara umum, semakin banyak gambar yang dilatihkan untuk tiap tiap kelas, sistem akan menjadi semakin baik dan andal ketika mengklasifikasikan gambar baru.

Ketika sistem telah dilatih dengan gambar yang cukup, selanjutnya sistem dapat diuji dengan memberikan gambar baru yang tidak diberikan pada saat pelatihan (*training*).



Kucing atau Anjing ?

Gambar 4.9 Pengujian dengan Gambar Baru

Sumber: vladimircech/Freeplik

Menurut kamu apa hasil klasifikasi dari gambar diatas? Jika hasil klasifikasinya keliru maka sistem *machine learning* kita dapat dilatih dan diuji kembali dengan gambar baru tadi, sama seperti manusia yang terus menerus belajar. Pelatihan/ pembelajaran yang terus menerus membuat sistem kita semakin pintar.

Tapi kita harus berhati-hati karena sistem kita hanya dirancang untuk hanya mampu mengklasifikasikan gambar yang telah kita latihkan, dalam hal ini adalah kucing atau anjing. Sebuah gambar yang sangat berbeda bisa jadi akan diklasifikasi sebagai kucing atau anjing. Sebagai contoh gambar kuda berikut, bisa saja terklasifikasi sebagai kucing atau anjing, yang merupakan klasifikasi yang salah. Jadi karena sistem pengklasifikasi kita hanya bisa membedakan dua kelas/kategori/label maka gambar berbeda akan diklasifikasi pada kedua kelas tersebut. Kelas dapat dikembangkan/ditambah untuk kelas/kategori yang lain dengan proses pelatihan dan pengujian kembali.



Kucing atau Anjing ?

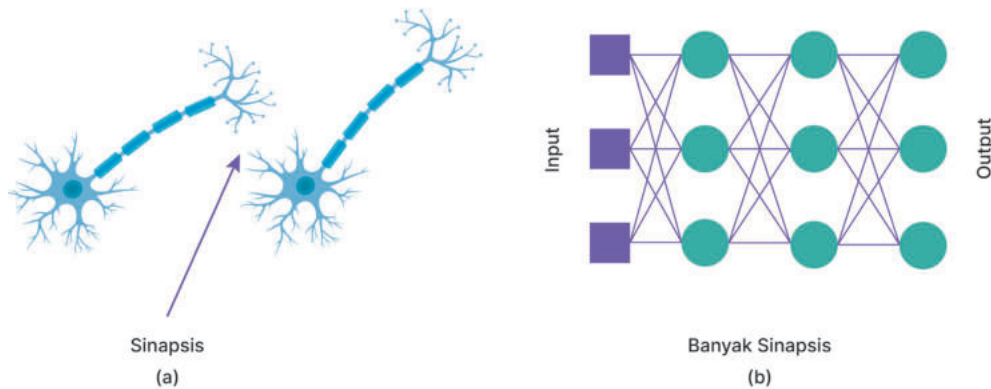
Gambar 4.10 Pengujian dengan Gambar yang Sangat Berbeda dari Kelas

Sumber: montypeter/Freeplik

Salah satu algoritma penting untuk klasifikasi gambar pada *Machine Learning* adalah *Artificial Neural Network* (Jaringan Saraf Buatan). *Artificial Neural Network* (ANN) diinspirasi dari cara kerja otak manusia yang terdiri dari kumpulan neuron. Pada sub berikut kamu akan belajar dasar dari algoritma ANN.

3. Artificial Neural Network

ANN adalah algoritma machine learning yang digunakan pada *library/extension* LookExtension MIT di IDE App Inventor. LookExtension akan kamu eksplorasi pada aktivitas selanjutnya. ANN memiliki cara untuk merepresentasikan pengetahuan dalam kumpulan *node* yang terinspirasi dari kumpulan neuron pada otak manusia yang saling tersambung. Hubungan antar *node* digambarkan dalam bentuk garis yang diilhami oleh sinapsis pada otak manusia. Pengetahuan pada *node* dan sinapsis tercipta melalui runtunan proses komputasi random, spesifik, dan saling terkait dengan *node* lain. Ilustrasi ANN sederhana tampak pada gambar berikut, *node* digambarkan dalam bentuk bulatan dan garis (hubungan antar *node*) digambarkan dengan garis panah.



Gambar 4.11 (a) Jaringan Otak Manusia, (b) Artificial Neural Network

Node input pada lapisan input menerima input, maka gambar akan direpresentasikan dalam bentuk data di *node input*, *node* berikutnya akan melakukan komputasi untuk menentukan kelas apa dari gambar pada input tersebut. Garis panah memiliki bobot/*weight* berbeda yang tampak dengan tebal tipisnya garis panah.

Proses komputasi yang terjadi dengan melibatkan bobot garis panah dan *node* terus berlanjut lapisan berikutnya yang akan dibandingkan dengan gambar sesuai label. Ada proses umpan balik (*feedback*) yang terjadi yang terus menerus, perbedaan perbandingan akan memicu perubahan bobot

pada garis panah sehingga membentuk konfigurasi yang optimal. Umpan balik akan berhenti saat konfigurasi telah dianggap optimal.

Proses pelatihan adalah proses untuk memperbaharui bobot pada garis panah menuju kondisi optimal yang terbaik untuk pengklasifikasian pola. Pelatihan dianggap cukup, jika pengklasifikasian telah mampu mengklasifikasinya gambar sesuai dengan labelnya, bahkan juga jika gambar adalah gambar baru.

Proses pelatihan adalah komputasi yang cukup kompleks seperti bagaimana menghitung *error* pada saat pembandingan, pembaharuan bobot, dan termasuk bagaimana gambar untuk pelatihan telah mewakili keseluruhan dari gambar yang akan diklasifikasi. Hal ini terus menjadi topik riset di *Machine Learning* dan ANN sampai saat ini.

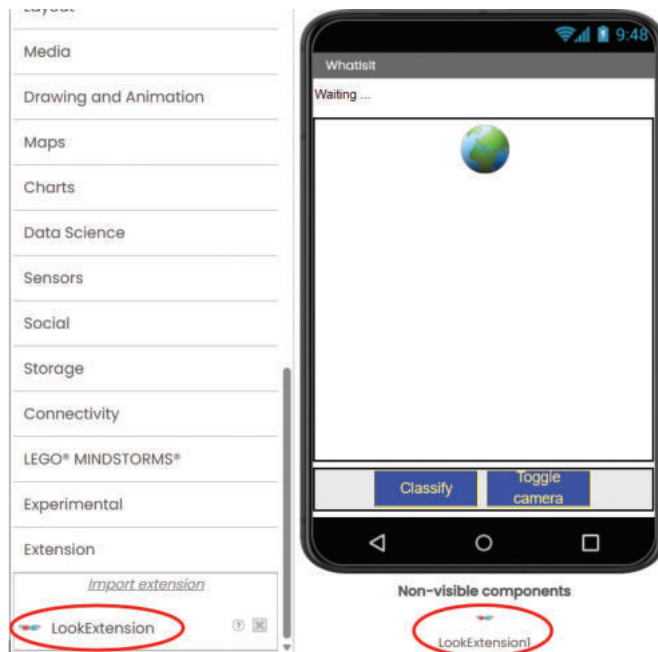
Pada ANN dikenal dua jenis pelatihan yang disebut *supervised* dan *unsupervised learning*. *Supervised learning* adalah cara umum yang digunakan untuk klasifikasi gambar, dimana gambar input dan label telah diketahui. Contoh pada klasifikasi gambar dengan kelas/kategori/label “kucing” dan “anjing”, adalah salah satu contoh *supervised learning*.

Jenis pelatihan *unsupervised learning* menggunakan cara dimana input tersedia, namun kelas/kategori/label belum diketahui. Pelatihan jenis ini biasanya digunakan untuk mencari pola baru, misalnya dari data perjalanan para turis, data medis yang akan dicari pola baru yang belum diketahui sebelumnya.

4. LookExtension

Ekstensi LookExtension adalah *library Neural Network* dengan jenis mobilenet yang secara khusus dirancang untuk mengenali gambar dengan menggunakan ponsel. Mobilenet sebenarnya telah dilatih untuk mengenali 999 kelas, dengan jutaan gambar. Kelas gambar tersebut dapat diakses di tautan <https://buku.kemdikbud.go.id/s/mjhnu4> termasuk dengan labelnya.

Ekstensi ini bukan merupakan kode inti dari App Inventor namun dapat digunakan dengan App Inventor dengan melakukan impor ekstensi. Tampilan LookExtension pada proyek “**Whatisit**” tampak pada gambar berikut:



Gambar 4.12 LookExtension pada Proyek Whatisi

Sumber: Dela Chaerani/Kemendikbudristek (2024)



Ayo Kembangkan

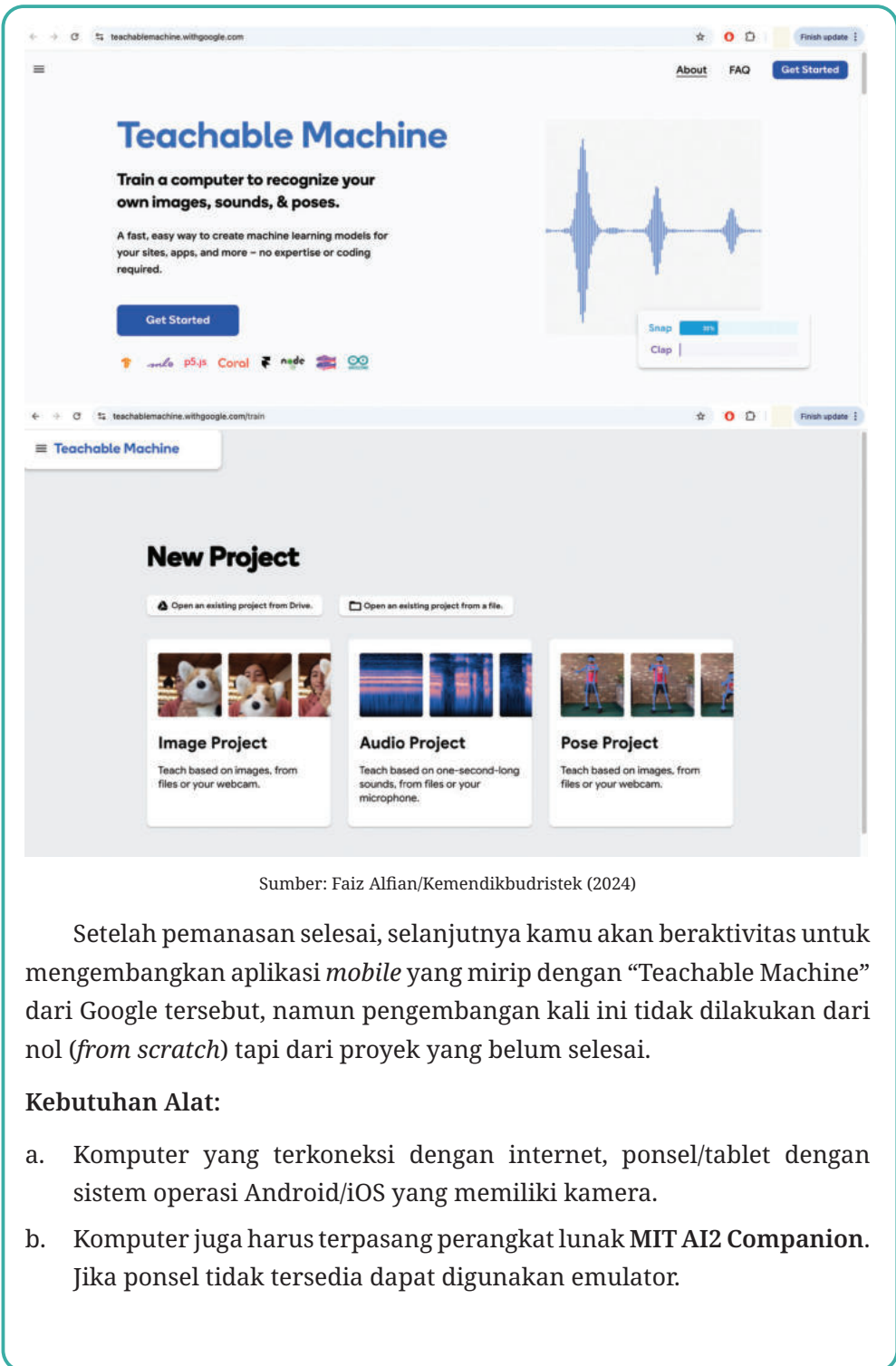
Image Classifier dengan App Inventor

Jenis Aktivitas: Kelompok

No Aktivitas: LD-K11-12

Aktivitas ini akan mengajak kamu untuk belajar dasar machine learning sebagai bagian dari Kecerdasan Buatan dengan membuat sendiri aplikasi mobile yang mampu menerapkan *machine learning* untuk mengklasifikasi gambar.

Sebagai pemanasan kamu diajak untuk bermain main terlebih dahulu dengan sistem *machine learning* dari Google yang dapat diakses pada situs berikut: <https://teachablemachine.withgoogle.com/>. “Teachable Machine” ini dapat mengklasifikasikan gambar (*image*), suara (*audio*), dan pose. Berikut tangkapan layar dari Teachable Machine. kamu akan dituntun oleh guru untuk melakukan pemanasan pengenalan gambar (*image recognition*) dengan teachable machine ini.



Sumber: Faiz Alfian/Kemendikbudristek (2024)

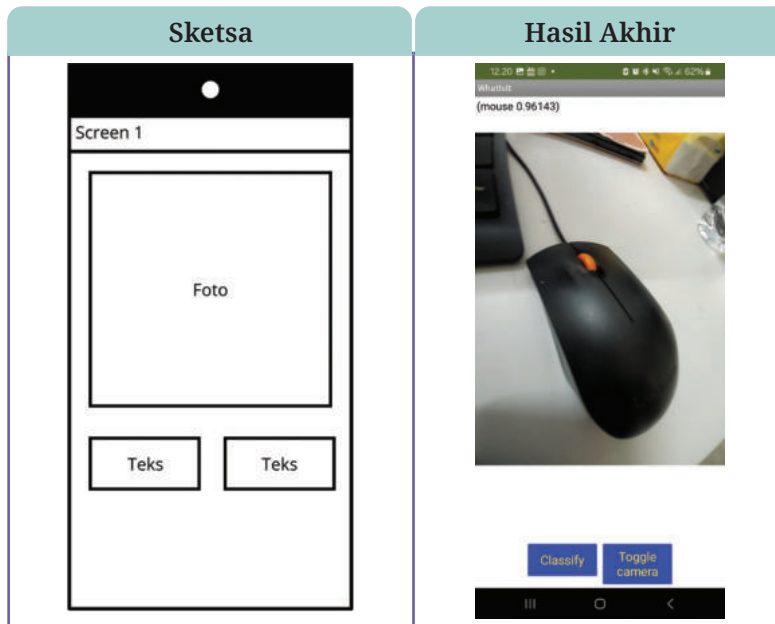
Setelah pemanasan selesai, selanjutnya kamu akan beraktivitas untuk mengembangkan aplikasi *mobile* yang mirip dengan “Teachable Machine” dari Google tersebut, namun pengembangan kali ini tidak dilakukan dari nol (*from scratch*) tapi dari proyek yang belum selesai.

Kebutuhan Alat:

- Komputer yang terkoneksi dengan internet, ponsel/tablet dengan sistem operasi Android/iOS yang memiliki kamera.
- Komputer juga harus terpasang perangkat lunak MIT AI2 Companion. Jika ponsel tidak tersedia dapat digunakan emulator.

Deskripsi Produk:

Kamu akan belajar membuat perangkat lunak berbasis *mobile* yang dapat mengklasifikasikan gambar dengan *LookExtension*. Perangkat lunak berfungsi dengan alur, jika pengguna memotret sebuah objek menggunakan kamera ponsel/tablet, maka informasi kelas/kategori dari objek yang dipotret tertampil di layar ponsel. Kelas/kategori ditentukan dengan tingkat kepercayaan tertentu. kamu akan mengembangkan aplikasi *mobile* yang memiliki antarmuka sebagai berikut:



Sumber: Dela Chaerani/Kemendikbudristek (2024)

Spesifikasi:

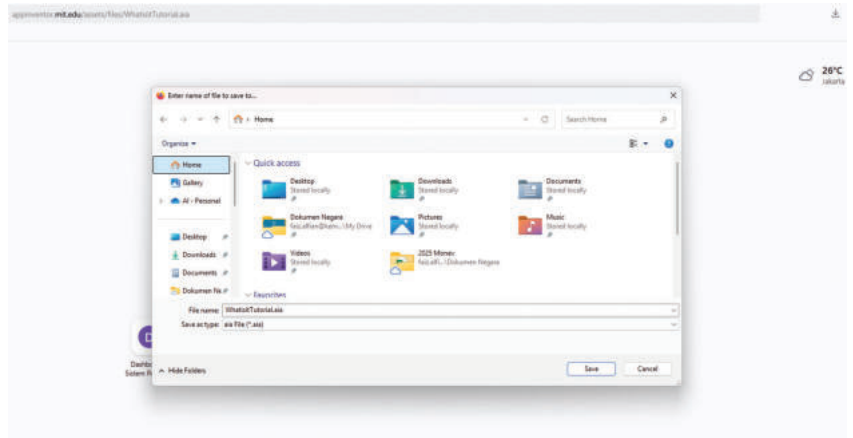
- **Input:** Pengguna mengarahkan kamera ponsel pada objek tertentu dan menekan *Classify* pada aplikasi
- **Proses:** Sistem akan mengklasifikasi objek
- **Output:** Pada gambar "Hasil Akhir", hasil klasifikasi gambar menunjukkan objek adalah Mouse, dengan tingkat kepercayaan 0.96143 (semakin mendekati 1, semakin yakin).

Sistem juga dilengkapi dengan fitur untuk menghidupkan kamera ponsel, dan mengubah kamera yang aktif, apakah kamera depan atau belakang.

Langkah Pengembangan:

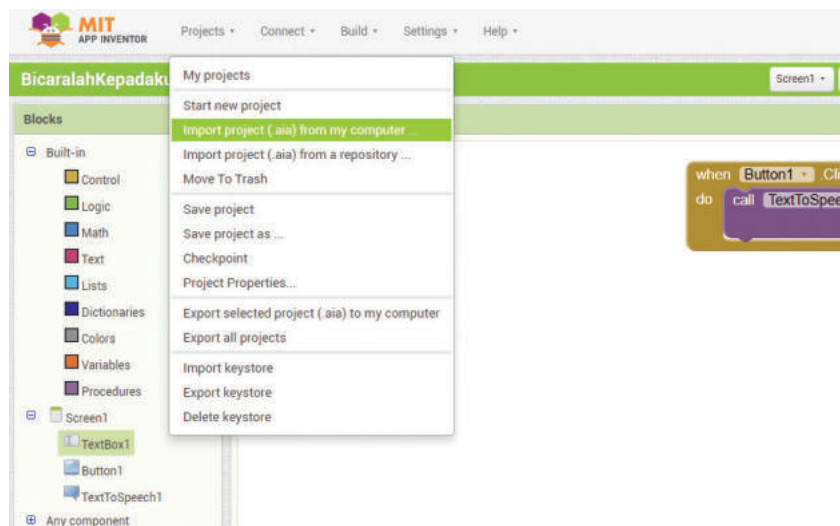
a. Persiapan:

- i. Download template aplikasi machine learning di <https://appinventor.mit.edu/assets/files/WhatisitTutorial.aia>



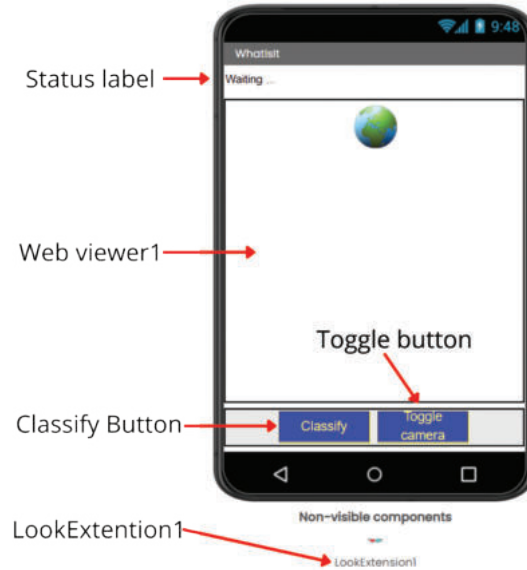
Sumber: Faiz Alfian/Kemendikbudristek (2024)

- ii. Import file hasil unduhan ke dalam IDE App Inventor sebagai proyek, dengan memilih menu *Import project (.aia) from my computer*.



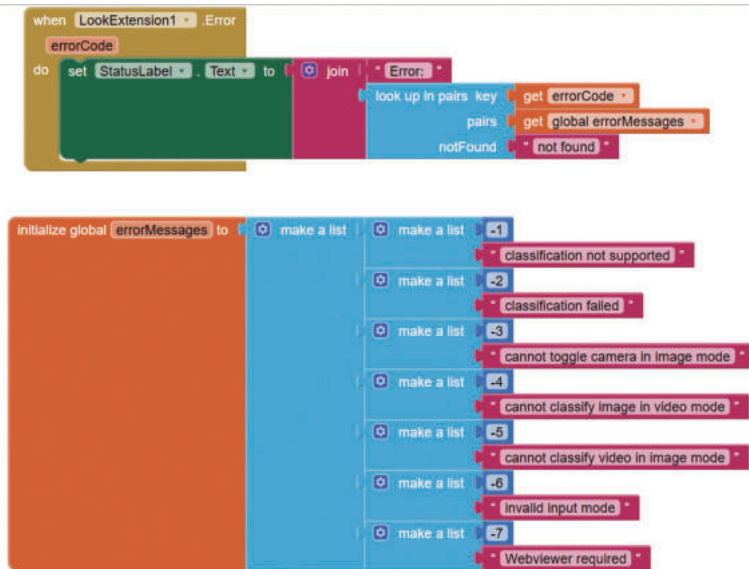
Sumber: Faiz Alfian/Kemendikbudristek (2024)

Tampilan dari proyek hasil impor tampak pada gambar berikut, perhatikan tampilan antarmuka pengguna, dan cari tahu apa kegunaan masing- masing komponen:



Sumber: Dela Chaerani/Kemendikbudristek (2024)

- iii. Eksplorasi kode yang tersedia, dengan melihat blok yang telah terdefinisi di editor blok. Ada beberapa blok telah tersedia, yaitu:

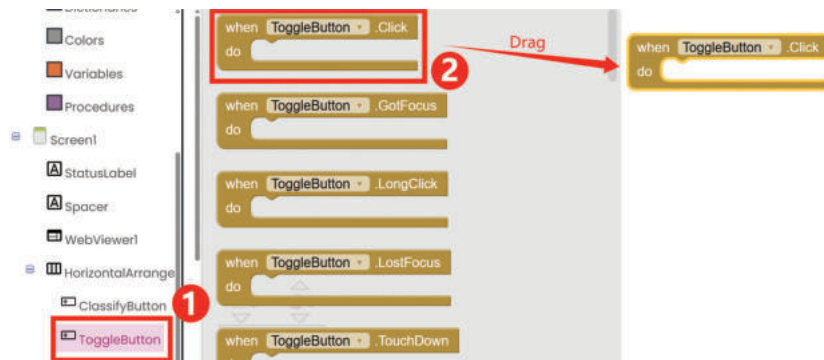


Sumber: Dela Chaerani/Kemendikbudristek (2024)

b. Pengkodean Program (Modifikasi Program):

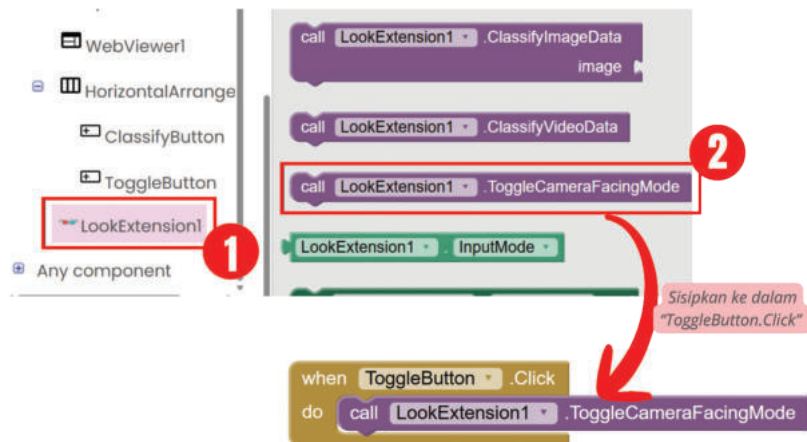
Modifikasi program dengan menambahkan kode blok sebagai berikut:

- i. Menambahkan kode button *ToggleButton* untuk *Toggle Camera*, dengan blok `when ToggleButton.Click`



Sumber: Dela Chaerani/Kemendikbudristek (2024)

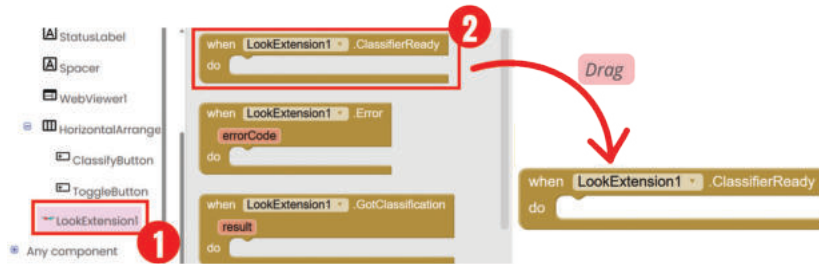
- ii. Isi `ToggleButton.Click` dengan blok `LookExtension1.ToggleCameraFacingDown`



Sumber: Dela Chaerani/Kemendikbudristek (2024)

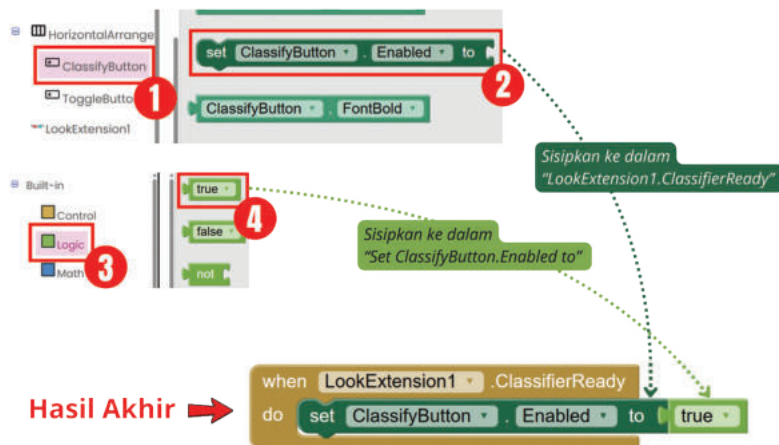
- iii. Mempersiapkan fungsi klasifikasi di ekstensi *LookExtension*: *LookExtension* sebagai *library* Pengklasifikasi Gambar berhubungan dengan beberapa komponen agar fungsi *classifier* dapat berfungsi dengan baik. Langkah untuk mempersiapkan *LookExtension* adalah:

- Drag and drop fungsi ClassifierReady dari komponen LookExtension1 ke kolom Viewer



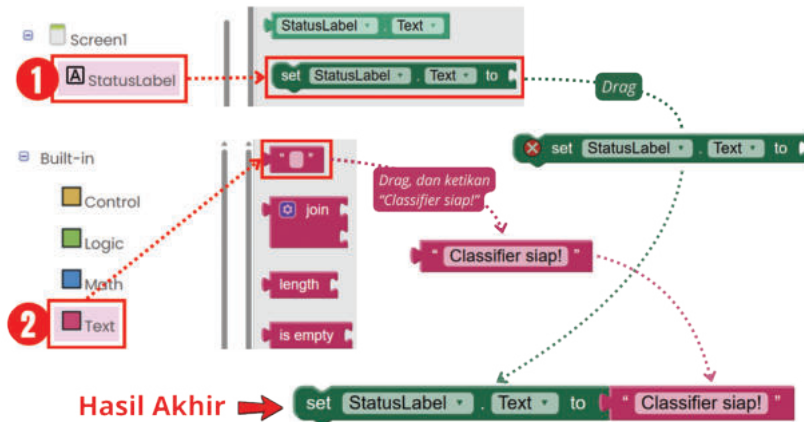
Sumber: Dela Chaerani/Kemendikbudristek (2024)

- Isi LookExtension1.ClassifierReady dengan mengatur ClassifyButton.Enabled menjadi True



Sumber: Dela Chaerani/Kemendikbudristek (2024)

- iv. Setelah langkah iii, *LookExtension* telah siap digunakan. Langkah berikutnya mengeset *StatusLabel* dengan teks “**Classifier Siap!**”.



Sumber: Dela Chaerani/Kemendikbudristek (2024)

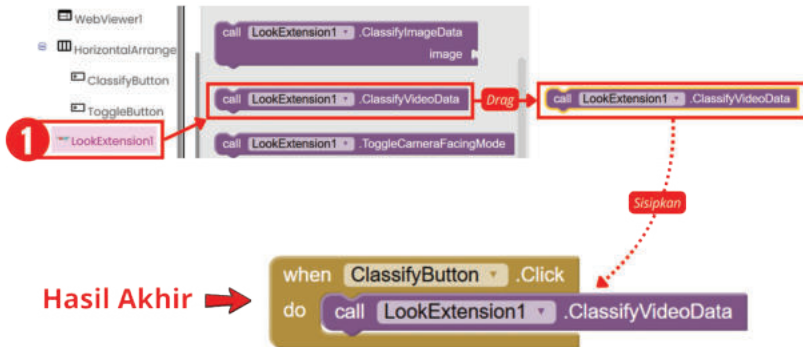
- v. Selanjutnya tambahkan blok kode untuk tombol *ClassifyButton*, yaitu:

- *Drag and drop* blok *when ClassifyButton.Click*



Sumber: Dela Chaerani/Kemendikbudristek (2024)

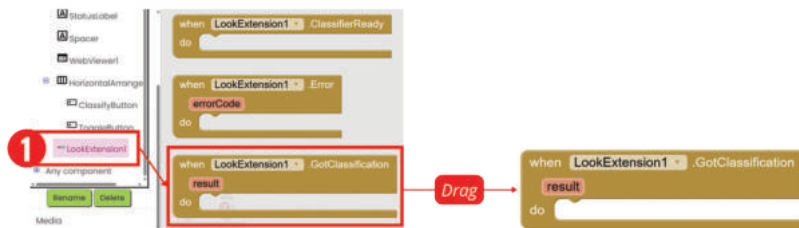
- Isikan blok `Call LookExtension1.ClassifyVideoData` ke blok `when ClassifyButton.Click`



Sumber: Dela Chaerani/Kemendikbudristek (2024)

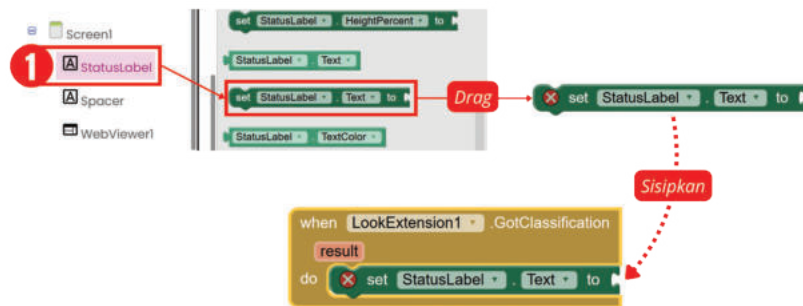
- Setelah fungsi klasifikasi pada `LookExtension` selesai di `set`, selanjutnya tambahkan kode untuk menampilkan hasil klasifikasi pada `StatusLabel`.

- Drag and drop blok `LookExtension.GotClassification`



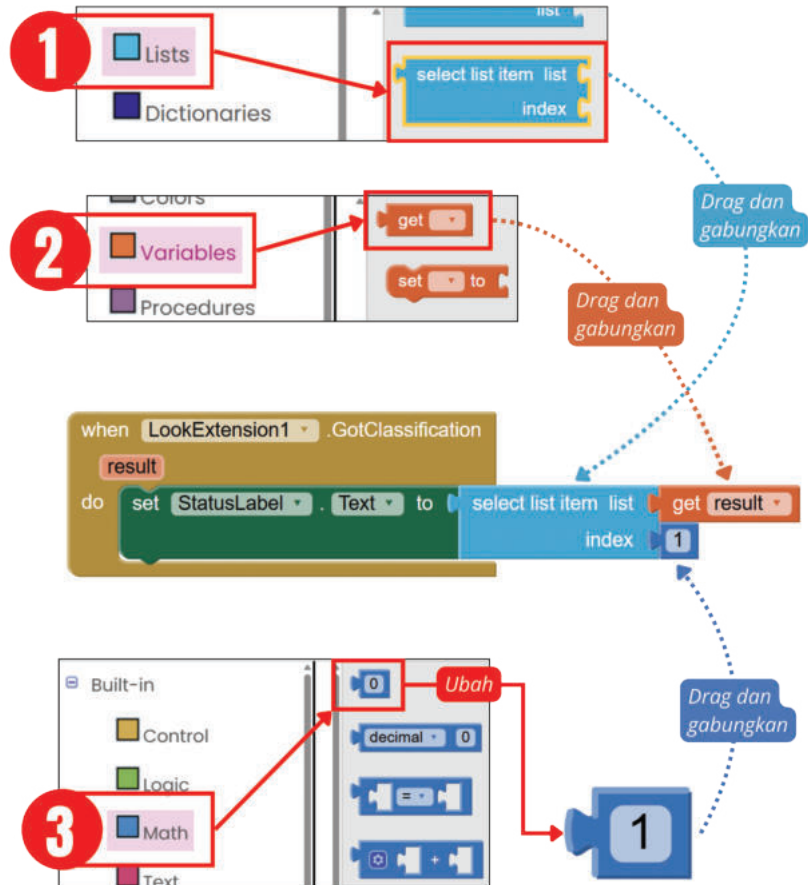
Sumber: Dela Chaerani/Kemendikbudristek (2024)

- Masukkan `StatusLabel.Text` ke dalam blok `LookExtension.GotClassification`



Sumber: Dela Chaerani/Kemendikbudristek (2024)

- Isi `StatusLabel.Text` dengan hasil dari klasifikasi yang berupa list of list, dengan format `[[klasifikasi1,akurasi1],[klasifikasi2,akurasi2],...,[klasifikasi10,akurasi10]]`, dimana `klasifikasi1` adalah klasifikasi dengan akurasi terbaik. Tarik *select list item index* dari komponen *built-in (lists)*, dan masukkan ke `StatusLabel.Text`, seperti pada gambar berikut:



Sumber: Dela Chaerani/Kemendikbudristek (2024)

c. Pengujian Sistem:

Setelah kode selesai disusun, uji aplikasi untuk mengklasifikasi beberapa objek dengan mengambil gambar dari beberapa sudut pengambilan seperti dari depan, samping, dan belakang.

Selanjutnya, setelah pengujian selesai. Isilah tabel pengujian berikut:

Diisi Saat Perencanaan			Diisi Setelah Pengujian		
No	Fitur	Dikerjakan Oleh	Sesuai dengan Spesifikasi		Keterangan Hasil Pengujian
			Ya	Tidak	
1	a. Mengaktifkan kamera ponsel:				
	Kamera Depan				
	Kamera Belakang				
2	b. Mengenali objek tertentu:				
	Mouse-pengambilan gambar menghadap ke depan				
	Mouse-pengambilan gambar menghadap samping				
	Botol Air-pengambilan gambar menghadap ke depan				
	(Gunakan objek lainnya)				

d. Ide Pengembangan:

- i. Aplikasi dapat dikembangkan dengan menggabungkan aktivitas LD-K11-10 dengan LD-K11-12 yang membuat hasil klasifikasi dari *classifier* menjadi suara.
- ii. Aplikasi dapat juga dikembangkan untuk menampilkan hasil klasifikasi untuk dua item terbaik. Jadi tidak hanya satu item hasil klasifikasi yang tertampil di layar tapi dua item.
- iii. Bagaimana kalau aktivitas LD-K11-12 dikembangkan untuk mengenali suara? Atau gambar saja (bukan video) seperti kode diatas?
- iv. Apa ide pengembangan aplikasi kamu?



Ayo Kembangkan

Kalkulator dengan Suara

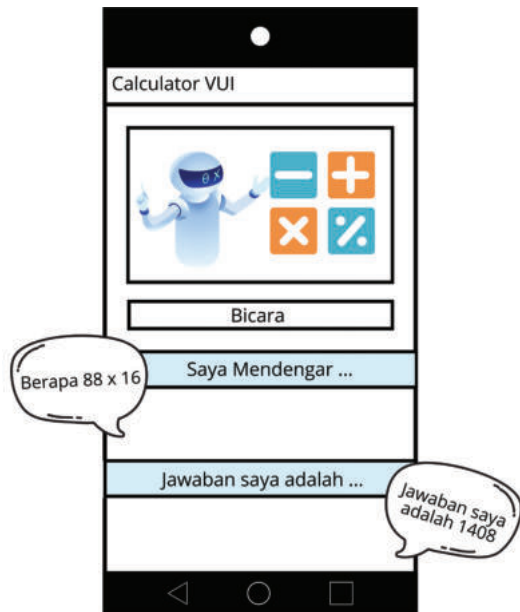
Jenis Aktivitas: Kelompok

No Aktivitas: LD-K11-13

Pernahkan kamu menggunakan pencarian dengan Google dengan suara? Atau bertanya dengan menggunakan kepada Alexa dan Siri? Bagaimana perangkat lunak tersebut menginterpretasi apa yang kita ucapkan? Dan bagaimana aplikasi tersebut merespon permintaan kita?

Tujuan dari pengembangan proyek Kecerdasan Buatan adalah untuk memberi pemahaman tentang dasar-dasar antarmuka pengguna berbasis suara (*Voice User Interface*) serta proses perancangan sistem Kecerdasan Buatan sederhana yang dapat memahami pengguna dalam pertanyaan dan tanggapan perhitungan yang dinyatakan secara lisan dengan tepat. Sistem Kecerdasan Buatan yang digerakkan oleh suara seperti ini dapat berguna dalam berbagai konteks seperti saat merancang teknologi bantu untuk penyandang disabilitas visual dan orang tua. Misalnya, pengguna tunanetra dapat menggunakan kalkulator suara untuk melakukan perhitungan matematis secara verbal tanpa harus mengetikkan semua detail perhitungan.

Aktivitas ini adalah aktivitas pengembangan perangkat lunak berbasis *Artificial Intelligence* menggunakan *library* yang telah ada di App Inventor dengan contoh desain layar pada gambar berikut.



Sumber: Dela Chaerani/Kemendikbudristek (2024)

Pada proyek ini kamu ditantang untuk membuat aplikasi yang mampu menggunakan *Voice User Interface* (VUI) untuk melakukan penghitungan aritmatika sederhana. Aplikasi yang dikembangkan mampu untuk:

- Melakukan interpretasi suara yang memerintahkan operasi aritmatika yaitu:
 - Penjumlahan (+)
 - Pengurangan (-)
 - Perkalian (x)
 - Pembagian (:)
- Menghitung operasi aritmatika tersebut dan menampilkan hasilnya di layar serta memperdengarkan dalam bentuk suara.

Catatan: Proyek ini tidak dapat menggunakan emulator pada pengujian karena aplikasi tergantung pada kemampuan pengenalan suara pada ponsel. Ponsel juga harus memiliki kemampuan pengenalan suara agar proyek dapat berfungsi.

Gunakan lembar kerja berikut untuk pengerjaan Kalkulator dengan suara ini.

Peran	Penanggung Jawab
Analisis Program:	
c. Deskripsi Produk	
d. Spesifikasi Aplikasi	
e. Kebutuhan resource: file, alat, dll	
Perancang <i>User Interface</i> (UI)	
Pemrogram Kode	
Penguji Program	
Pemapar Presentasi	

Spesifikasi (Deskripsi Produk, Fungsionalitas Aplikasi, Kebutuhan Resource)

Rancangan *User Interface* (UI)

Kode Program

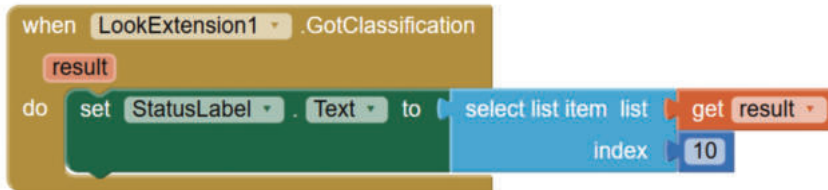
Pengujian

Diisi Saat Perencanaan			Diisi Setelah Pengujian		
No	Fitur	Dikerjakan Oleh	Sesuai dengan Spesifikasi		Keterangan Hasil Pengujian
			Ya	Tidak	

D. Uji Kompetensi

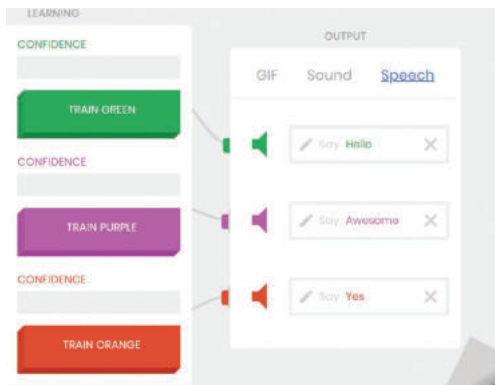
Soal Pilihan Ganda

1. Jika pada LookExtension1 kalian mengubah nilai kode index dari 1 ke 10. Apa yang akan terjadi ?



Sumber: Dela Chaerani/Kemendikbudristek (2024)

- a. Menampilkan 10 klasifikasi terbaik
- b. Terjadi pengulangan 10 kali untuk mendapatkan klasifikasi terbaik
- c. Menampilkan pesan kesalahan
- d. Tidak terjadi apa apa
2. Pada Teachable Machine, jika anda melatih sasando dengan warna hijau, angklung dengan warna *purple*, dan gamelan dengan warna *orange*. Dan kalian menguji dengan gambar gamelan. Suara apa yang akan terdengar?



- a. *Hello*
- b. *Awesome*
- c. *Yes*
- d. Tidak keluar suara apapun

Soal Uji Kompetensi

Buatlah sebuah kelompok, kemudian buat sebuah aplikasi menggunakan AppInventor untuk menghitung (BMI) *Body Mass Index*. Aplikasi ini harus dapat menampilkan kategori BMI dari hasil perhitungan pada program.

Langkah-langkah Pengerjaan:

1. Buatlah sebuah desain antarmuka pengguna
2. Buat fungsi perhitungan BMI sebagai berikut:
 - a. BMI dapat dihitung menggunakan rumus: $BMI = \text{Berat Badan (kg)} / (\text{Tinggi Badan (m)} * \text{Tinggi Badan (m)})$.
 - b. Buat *event handler* untuk tombol "Hitung BMI" yang mengambil nilai dari *TextBox*, menghitung BMI, dan menampilkan hasilnya di *Label*.
3. Tampilkan kategori BMI:
 - a. $BMI < 18.5$: *Underweight*
 - b. $18.5 \leq BMI < 24.9$: *Normal Weight*
 - c. $25 \leq BMI < 29.9$: *Overweight*
 - d. $BMI \geq 30$: *Obese*
4. Uji coba program kamu.
5. Pelaporan:
 - a. Kirimkan file .aia dari proyek yang kamu kerjakan
 - b. Kirimkan tangkap layar tampilan utama dan hasil perhitungan BMI
 - c. Jika ada, sertakan dokumentasi dari bagaimana program kamu bekerja
6. Poin bonus:
 - a. Terdapat fitur menyimpan riwayat perhitungan BMI
 - b. Tambahkan tombol untuk reset nilai masukan dan hasil perhitungan.